



**ARIA: An Adaptive Reforestation Intelligence Agent Using Reinforcement Learning for
Drone-Based Aerial Seed Placement in Rwanda**

BSc. in Software Engineering

Submitted by: Ernest Ntezirizaza
African Leadership University (ALU)

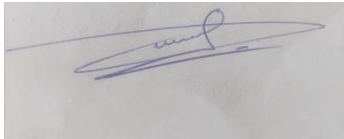
Supervisor: Dirac Murairi

July 2026

DECLARATION

I, Erneste Ntezirizaza, declare that this Final Capstone Report, titled "ARIA: An Adaptive Reforestation Intelligence Agent Using Reinforcement Learning for Drone-Based Aerial Seed Placement in Rwanda," is my original work, prepared under the supervision of Dirac Murairi at the African Leadership University. Except where explicitly stated and referenced, the ideas, analysis, design, implementation, and results presented in this report are my own, and all external sources, including academic literature, datasets, tools, and frameworks, have been properly cited and acknowledged in the References section of this document.

I further declare that this work has not been submitted, in whole or in part, for the award of a degree, diploma, or any similar qualification at this or any other university or institution of higher learning. I take full responsibility for the accuracy and integrity of the content presented, including the honest reporting of both the achievements and the limitations of this project.



Signature: Date: 24th July 2026

Erneste Ntezirizaza

CERTIFICATION

The undersigned certifies that he has read and hereby recommends for acceptance by African Leadership University a report entitled "ARIA: An Adaptive Reforestation Intelligence Agent Using Reinforcement Learning for Drone-Based Aerial Seed Placement in Rwanda," submitted by Ernest Ntezirizaza in partial fulfilment of the requirements for the degree of Bachelor of Science in Software Engineering. The undersigned supervised this work from its initial proposal through to completion, confirms that it reflects the student's own original research and analysis, and certifies that it meets the academic standard required by the university for acceptance and examination.

Signature: Date:

Mr. Dirac Murairi

Faculty, Bachelor of Software Engineering, ALU

DEDICATION AND ACKNOWLEDGEMENT

First and foremost, I am grateful to God for granting me the strength, wisdom, and endurance to complete my undergraduate studies and bring this capstone project to a successful conclusion.

I would like to sincerely appreciate my supervisor, Mr. Dirac Murairi, for the guidance and critical feedback that shaped this work from the very first project proposal through to its completion, and for consistently pushing me to think more rigorously about what my results actually show.

To my family, thank you for your unwavering love, prayers, and encouragement throughout this journey. Your support carried me through the most demanding stretches of this project, and I dedicate this work to you.

I also want to appreciate the facilitators and the entire faculty of the Software Engineering program at African Leadership University for the knowledge and mentorship provided over the course of my studies, as well as my fellow students and friends, whose companionship, encouragement, and shared late nights made this journey far more memorable.

ABSTRACT

In 2024, Rwanda lost 910 hectares of natural forest (Global Forest Watch, 2024), even as national forest cover grew from 10.7% to 30.4% between 2010 and 2022 (IISD, 2024). Existing aerial reforestation systems plan a fixed flight path before takeoff, or train in synthetic environments with no real soil, rainfall, or protected-area data.

This project set out to design, build, and evaluate ARIA, a reinforcement learning drone agent trained on six real Rwanda geospatial datasets to make terrain-adaptive seed placement decisions. The agent uses Proximal Policy Optimisation with a CNN terrain feature extractor inside a custom Gymnasium environment, and the trained policy was deployed through a Unity WebGL simulation backed by a live web application and database.

A five-configuration spacing-penalty sweep found a peak mean training reward of 2274.39, and the best-performing policy was then tested across six Rwanda terrain zones held out from training. Measured against each zone's achievable seeding ceiling, the policy placed between 41.9% and 60.5% of its available seeds into ecologically suitable cells, with no protected-area violations in any zone. Spacing violations stayed high, the clearest weakness in the results. In production, the deployed system recorded 1,182 seeds placed and an average reward of 96.42 across 12 live episodes. The reward-component ablation and baseline comparison planned at the outset were not finished within the eight-week timeline. In conclusion, the agent generalises well to unseen terrain and behaves correctly once deployed, with spacing penalty and missing baseline comparisons as priorities for future work.

Keywords: *Reinforcement Learning, PPO, Drone Reforestation, Rwanda, Geospatial AI, Aerial Seed Placement, Coverage Path Planning, MDP*

TABLE OF CONTENTS

List of Tables

List of Figures

List of Acronyms/Abbreviations

DECLARATION

CERTIFICATION

DEDICATION AND ACKNOWLEDGEMENT

ABSTRACT

CHAPTER ONE: INTRODUCTION

- 1.1 Introduction and Background
- 1.2 Problem Statement
- 1.3 Project's Main Objective
 - 1.3.1 List of the Specific Objectives
- 1.4 Research Questions
- 1.5 Project Scope
- 1.6 Significance and Justification
- 1.7 Research Budget
- 1.8 Research Timeline

CHAPTER TWO: LITERATURE REVIEW

- 2.1 Introduction
- 2.2 Historical Background of the Research Topic
- 2.3 Overview of Existing Systems
- 2.4 Review of Related Work
 - 2.4.1 Summary of Reviewed Literature
- 2.5 Strengths and Weaknesses of Existing Systems
- 2.6 General Comment and Conclusion

CHAPTER THREE: SYSTEM ANALYSIS AND DESIGN

- 3.1 Introduction
- 3.2 Research Design
 - 3.2.1 Dataset and Dataset Description
 - 3.2.2 Zone-Based Dataset Partitioning
 - 3.2.3 Dataset Preprocessing and Feature Extraction

3.2.4 Proposed Model Diagram

3.2.5 Expected Results and Hypothesis

3.3 Functional and Non-Functional Requirements

3.4 System Architecture

3.5 Flow Chart, Class Diagram, Use Case Diagram, Sequence Diagram and All Other Diagrams

3.6 Development Tools

3.7 Ethical Considerations

CHAPTER FOUR: SYSTEM IMPLEMENTATION AND TESTING

4.1 Implementation and Coding

4.1.3 Key Modules and Source Code

4.2 Graphical View of the Project

4.3 Testing

CHAPTER FIVE: RESULTS & DISCUSSION

5.1 Spacing-Penalty Ablation Results

5.2 Generalisation to Held-Out Zones

5.3 Production Dashboard Results

5.4 Results Mapped to Research Questions

5.5 Summary

CHAPTER SIX: CONCLUSIONS AND RECOMMENDATIONS

6.1 Introduction

6.2 Conclusions

6.3 Limitations

6.4 Recommendations for Future Research

References

LIST OF TABLES

Table 1.1: Research Budget

Table 1.2: Project Implementation Timeline

Table 2.1: Summary of Reviewed Literature

Table 3.1: Rwanda Geospatial Datasets

Table 3.2: Ablation Study Conditions

Table 3.3: Evaluation Metrics

Table 3.4: Functional Requirements

Table 3.5: Non-Functional Requirements

Table 3.6: Algorithm Comparison

Table 3.7: Development Tools (As Planned)

Table 4.1: Interface Compliance with Functional Requirements

Table 4.2: Unit Testing Outputs

Table 4.3: Validation Testing Outputs

Table 4.4: Integration Testing Outputs

Table 4.5: Functional and System Testing

Table 4.6: Acceptance Testing Report

Table 5.1: Peak Mean Reward by Spacing-Penalty Configuration

Table 5.2: Generalisation Evaluation Across Six Held-Out Zones

Table 5.3: Production Dashboard Metrics

Table 5.4: Research Questions and Empirical Answers

LIST OF FIGURES

Figure 3.1: ARIA Ecological Input Layers Across a Representative Rwanda Zone

Figure 3.2: Correlation Between Ecological Variables

Figure 3.3: Pairwise Relationships Between Ecological Variables

Figure 3.4: Distribution of Ecological Variables Across All Sampled Zones

Figure 3.5: Composite Zone Suitability Map

Figure 3.6: ARIA Raw Dataset Summary

Figure 3.7: Proposed Model Architecture

Figure 3.8: ARIA System Architecture

Figure 3.9: Episode Execution Flow Chart

Figure 3.10: Use Case Diagram

Figure 3.11: Class Diagram

Figure 3.12: Sequence Diagram

Figure 3.13: Entity Relationship Diagram (Crow's Foot Notation)

Figure 5.1: Cumulative Reward by Experiment

Figure 5.2: Convergence Comparison Across the Five Experiments

Figure 5.3: Policy Entropy Curves

Figure 5.4: Generalisation Metrics Across Six Held-Out Rwanda Terrain Zones

Figure 5.5: Seeding Efficiency — Actual vs. Achievable Ceiling per Zone

Figure 5.6: Species Recommender — Learned Feature Weights from Reseed Outcomes

LIST OF ACRONYMS/ABBREVIATIONS

Acronym	Full Meaning
ARIA	Adaptive Reforestation Intelligence Agent
RL	Reinforcement Learning
PPO	Proximal Policy Optimisation
MDP	Markov Decision Process
UAV	Unmanned Aerial Vehicle
DEM	Digital Elevation Model
ICRAF	World Agroforestry Centre
IUCN	International Union for Conservation of Nature
GFW	Global Forest Watch
NST2	National Strategy for Transformation 2 (Rwanda, 2024–2029)
SB3	Stable-Baselines3
FLR	Forest Landscape Restoration
W&B	Weights & Biases
WDPA	World Database on Protected Areas
ONNX	Open Neural Network Exchange
CHIRPS	Climate Hazards Group InfraRed Precipitation with Stations
CPP	Coverage Path Planning
REST	Representational State Transfer
API	Application Programming Interface
RFA	Rwanda Forestry Authority
SDLC	Software Development Life Cycle
ERD	Entity Relationship Diagram
ORM	Object-Relational Mapping
CNN	Convolutional Neural Network

CHAPTER ONE: INTRODUCTION

1.1 Introduction and Background

Forests regulate climate, shelter biodiversity, and sustain millions of livelihoods across sub-Saharan Africa. Yet forest cover keeps declining because of agricultural expansion, charcoal production, and population growth (AFR100, 2025). Rwanda shows this tension clearly. Forest cover rose from 10.7% in 2010 to 30.4% in 2022 through sustained national policy (IISD, 2024), yet 910 hectares of natural forest were still lost in 2024 alone, releasing roughly 500 kilotons of CO₂ equivalent (Global Forest Watch, 2024; The Forefront Magazine, 2025). Between 2009 and 2019, Rwanda lost 105,713 hectares of forest, a 15.7% decline in one decade (UNEP, 2023). Eastern Province loses forest at 2% annually, and about 85% of national energy demand is still met by biomass (UNEP, 2023; AFR100, 2025).

That recovery did not happen by accident. Rwanda's forest cover growth traces back to a deliberate sequence of policy interventions: the 2004 National Land Use Policy that restricted agricultural expansion onto marginal and steep land, the Green Growth and Climate Resilience Strategy of 2011, and the district-level Imihigo performance contracts that gave local leaders binding tree-planting targets tied to their own evaluations. NST2 2024–2029 continues this line, setting forest cover and landscape restoration as an explicit priority alongside digital transformation (Government of Rwanda, 2024). The pattern that runs through all of these policies is that reforestation in Rwanda has mostly been driven by land-use rules and planting quotas, not by any system that decides, cell by cell, where a seed actually belongs.

The drivers behind the remaining losses are not just clearing for its own sake. Subsistence agriculture on hillsides pushes into forest margins as population density rises, especially in the Southern and Western provinces where arable land is already scarce. Charcoal and firewood demand, which still covers most rural cooking energy, keeps a steady extraction pressure on woodlots near population centers. Eastern Province's 2% annual loss is compounded by cattle corridors and land pressure from resettlement, which is part of why it is treated as a harder test zone later in this report (AFR100, 2025). None of these pressures are solved by planting faster. They are solved, in part, by making every seed that does get planted more likely to survive, which is the argument for the terrain-adaptive placement problem this project addresses.

The stakes go beyond a carbon accounting line. Forest loss in Rwanda's hill country directly worsens erosion and landslide risk on the same steep slopes where population density is highest, which in turn threatens the water catchments that Kigali and secondary cities depend on. Reforestation and livelihoods are tied together here: agroforestry species selection affects fuelwood availability, soil fertility for adjacent farmland, and in some districts direct income from fruit or timber species. NST2's climate resilience pillar is explicit about this link between forest

cover and both water security and rural income (Government of Rwanda, 2024), which is why species-site matching is not a purely ecological concern, it is also a livelihoods one.

Traditional reforestation relies on manual planting, which is limited to accessible terrain and far too slow for Rwanda's 65-million-seedling target for 2024–2025 (IUCN, 2025). Software-assisted drone seeding is a real step forward here. AirSeed Technologies in Australia disperses 40,000 seed pods a day, and Brazil's MORFO fires 180 capsules a minute. Both are reviewed in more depth in Chapter Two. What both share is a planning process that happens entirely before takeoff, with no way to adjust once the drone is in the air. Seeds still land on incompatible soils, on slopes too steep for germination, or in rainfall zones that cannot sustain the target species, and field studies put survival rates for aerial dispersal as low as 0–20% (Robinson et al., 2022).

Reinforcement learning (RL) offers a different approach to this specific piece of the problem. An RL agent learns placement decisions sequentially through reward feedback, so it can adapt to terrain conditions at each step instead of committing to a fixed flight plan. Rwanda's six agro-ecological zones are already well characterised by open-access datasets (ICRAF, 2022), which gives a training foundation that most countries in the region do not yet have assembled in one place. ARIA was built on that foundation, aligning with Rwanda's NST2 2024–2029 priorities without being a policy document itself (Government of Rwanda, 2024).

1.2 Problem Statement

The first problem is ecological mismatch at the seed placement level. Rwanda's 115 validated tree species across six agro-ecological zones each need specific soil types, rainfall thresholds, altitude ranges, and slope tolerances (ICRAF, 2022). No existing aerial seeding system matches seeds to terrain requirements in real time. Rwanda's FLR Strategy 2024–2029 calls for data-driven, species-site-specific decisions (AFR100, 2025), yet no deployed software system delivers that. The result is 0–20% aerial seeding survival in field conditions (Robinson et al., 2022). At Rwanda's scale of 65 million seedlings a year (IUCN, 2025), a 10% improvement through intelligent placement means 6.5 million additional trees a season.

The second problem is physical inaccessibility combined with scale. Eastern Province loses 2% of its forest every year (AFR100, 2025) and has terrain that human planting teams cannot reach on foot. Drones can reach it, but flying a fixed grid over ecologically unsuitable terrain wastes every seed dropped there. The problem is not drone hardware. What is missing is an intelligent system that decides, in real time, the best placement location for each seed given local terrain conditions.

The third problem is that deployed aerial reforestation software largely plans once and never adjusts. Every commercial system reviewed in this study finalises its flight path before takeoff and executes it without adjustment; emerging vision-guided systems that classify terrain onboard exist, but none published to date combine that in-flight adaptation with a trained reinforcement learning policy. Siedler (2022) showed that reinforcement learning can produce reforestation coverage

policies, but only in a fully synthetic environment with no real terrain, soil, species, or protected area data behind it. What is still missing is a system that combines reinforcement learning with real African geospatial data for this specific task, rather than a synthetic proxy for it. This work introduced ARIA, a reinforcement learning system trained on six real Rwanda geospatial datasets, built to address all three problems through terrain-adaptive, ecologically informed sequential seed placement.

Reinforcement learning is not the only way to close this gap, and it is worth being explicit about why it was chosen over a non-learning alternative such as constrained optimisation solved fresh for each zone. On a static, fully observable grid, a well-tuned optimiser could plausibly match a trained policy's performance on the zone it was tuned for. The property that a re-optimised solution does not have, and a trained policy does, is generalisation: once ARIA's policy is trained, it can be dropped into a new, unseen terrain zone and produce a placement strategy immediately, without being re-solved from scratch for that zone's specific soil, rainfall, and slope profile. Given that Rwanda's FLR strategy calls for this kind of decision-making across many zones, not just one, that generalisation property is the specific reason reinforcement learning was chosen over a classical optimisation approach here.

1.3 Project's Main Objective

To develop ARIA, a reinforcement learning simulation system that trains an autonomous drone agent to make terrain-adaptive seed placement decisions across Rwanda's degraded land using six real geospatial datasets, in order to maximise ecological suitability coverage and validate the resulting policy against held-out terrain and a live production deployment.

1.3.1 List of the Specific Objectives

- a.** To review RL-based UAV path planning and aerial reforestation literature across four indexed academic platforms, and to construct a validated Rwanda-specific Gymnasium simulation environment (RwandaReforestEnv) from six publicly available geospatial datasets (DEM, soil classification, CHIRPS v3 rainfall, land cover, WDPA protected areas, and ICRAF species suitability), verified using `gym.utils.env_checker`, by end of Week 2.
- b.** To design and implement the unified ARIA planner-and-navigator policy, a PPO-based agent with a CNN terrain feature extractor and a six-component terrain-adaptive reward function, and to evaluate the effect of the seed-spacing penalty on training reward and convergence through a controlled five-configuration parameter sweep, by end of Week 5.
- c.** To implement the supporting web platform and Unity simulation, comprising a Postgres-backed telemetry system, a REST API, a web dashboard, and a WebGL-deployed drone simulation running the trained policy live via ONNX inference through Unity's Inference Engine, by end of Week 7.

d. To evaluate the best-performing policy's generalisation across six held-out Rwanda terrain zones using ecological suitability metrics measured against each zone's achievable seeding ceiling, and to validate the deployed policy's behaviour against real production telemetry, by end of Week 8.

1.4 Research Questions

I. To what extent does the seed-spacing penalty weight affect the trained policy's training reward and convergence behaviour?

II. Does the trained policy generalise to Rwanda terrain zones held out from training, and how does its seed-placement coverage compare against each zone's achievable seeding ceiling?

III. Does the trained policy behave correctly and consistently once deployed to a live production environment, compared to its behaviour during offline training and evaluation?

1.5 Project Scope

This project focused on the design, development, deployment, and evaluation of ARIA, an adaptive reforestation intelligence agent that trains a reinforcement learning drone to make terrain-adaptive seed placement decisions across Rwanda's degraded land. The study included the construction of a Rwanda-specific Gymnasium simulation environment from six real geospatial datasets, development and evaluation of a unified PPO+CNN planner-and-navigator policy through a five-configuration spacing-penalty sweep, generalisation evaluation of the best-performing policy across six held-out terrain zones measured against each zone's achievable seeding ceiling, and deployment of the trained policy to a live Unity WebGL simulation with a Postgres-backed telemetry dashboard accessible at aria-capstone.vercel.app. System performance was evaluated using procedurally generated terrain zones built from real Rwanda geospatial data and live production telemetry, as collecting continuous multi-season field data from an operational drone was beyond the scope of this study.

The project was limited to prototype development and evaluation rather than full-scale, real-world deployment. Several things fall outside this scope. No physical drone hardware was built or flown; all flight and seeding behaviour was simulated rather than tested in the field. The evaluation covered 42 modelled terrain zones rather than the whole of Rwanda. All geospatial inputs, such as soil and rainfall data, were procedurally generated rather than drawn from live satellite or soil-sensor feeds. Finally, the project did not extend to coordinating multiple drones at once or to long-term, multi-season field testing in protected areas.

1.6 Significance and Justification

Upon successful implementation, ARIA will directly benefit Rwanda's reforestation programme. At 65 million seedlings per year (IUCN, 2025), a 10% improvement in placement survival yields

6.5 million additional trees per season. This supports carbon sequestration, watershed protection, and NST2 progress (Government of Rwanda, 2024).

All six ARIA datasets are open-licensed and globally available. Researchers in Kenya, Uganda, Tanzania, or DRC could substitute a national DEM and local species database and replicate the framework within days, which makes ARIA a replicable continent-scale methodology rather than a Rwanda-specific one-off.

ARIA connected Rwanda's two NST2 priorities, climate resilience and digital innovation (Government of Rwanda, 2024). As an open-source proof of concept with a deployable web demo, it gave the Rwanda Forestry Authority an immediately evaluable reference architecture for technology-led reforestation.

1.7 Research Budget

Item	Description	Cost (USD)
Six Rwanda geospatial datasets	NLA/UNDP, ISRIC, CHC, GFW, WDPA, ICRAF (all open-access)	\$0.00
Python libraries (SB3, Gymnasium, rasterio, web frameworks)	Open-source, pip installable	\$0.00
Unity engine	3D terrain visualisation	\$0.00
Weights & Biases	Experiment tracking + sweeps	\$0.00
Hosting	Web application + database deployment	\$0.00
Cloud GPU	Google Colab Pro for W&B sweep if local CPU insufficient	\$20.00
Additional Colab compute units	Extra compute credits for extended training runs	\$10.00
Internet and data downloads	330 MB datasets	\$5.00
TOTAL		\$35.00

Table 1.1: Research Budget

1.8 Research Timeline

Table 1.2 presents the eight-week ARIA implementation timeline, organised into four phases and expressed in working days rather than fractional weeks. The Foundation phase (Weeks 1–3) covers literature review, dataset acquisition, and construction of the RwandaReforestEnv Gymnasium environment. The RL Core phase (Weeks 3–4) builds the unified PPO+CNN planner-and-navigator policy, the single model that makes both mission-level decisions (which zone, when to return, when to abort) and step-level decisions (where to fly, which species to drop) together. The

Simulation and Training phase (Weeks 4–6) implements the energy, weather, obstacle-avoidance, and animal-disturbance systems alongside the seed monitoring and reseedling pipeline, and runs the PPO training itself. The Web and Evaluation phase (Weeks 7–8) covers the real-time telemetry dashboard, the Unity WebGL deployment, evaluation on the held-out zones, and the final report.

#	Task	Days	W1	W2	W3	W4	W5	W6	W7	W8
PHASE 1: FOUNDATION (Weeks 1–3)										
1	Literature review across 4 academic platforms (22 papers)	5	■							
2	Acquire & preprocess 6 Rwanda geospatial datasets (DEM, soil, rainfall, land cover, WDPA, species)	5		■						
3	Build & validate RwandaReforestEnv (gym.utils.env_checker)	5			■					
PHASE 2: UNIFIED PLANNER + NAVIGATOR (RL CORE)										
4	Design 6-component reward function (soil, rain, slope, spacing, protected-area, step penalty)	3			■					
5	Implement unified PPO+CNN policy: single forward pass for mission-level (zone/return/abort) + step-level (fly/species) decisions	8				■	■			
PHASE 3: SIMULATION SYSTEMS + TRAINING										
6	Energy system (solar/battery), weather system + rain cover mechanism	3					■			
7	Obstacle detection/avoidance + animal disturbance engine	3					■			
8	Seed monitoring & automatic reseedling pipeline	3						■		
9	Spacing-penalty ablation: 5 configs × 200k timesteps	5						■		
PHASE 4: WEB, TELEMETRY DASHBOARD & EVALUATION										
10	Postgres schema + telemetry write pipeline + real-time dashboard	3							■	
11	Unity WebGL build + ONNX inference deployment (aria-capstone.vercel.app)	3							■	
12	Generalisation evaluation across 6 held-out zones (efficiency vs. achievable ceiling)	3								■

#	Task	Days	W1	W2	W3	W4	W5	W6	W7	W8
13	Final report write-up & submission	3								■

Table 1.2: Project Implementation Timeline

CHAPTER TWO: LITERATURE REVIEW

2.1 Introduction

This chapter reviews the literature underpinning the development of ARIA. It examines the evolution of drone-based technology in reforestation, the principles of reinforcement learning for autonomous coverage path planning, existing aerial seeding systems and their limitations, machine learning approaches to UAV navigation and terrain adaptation, and the ecological and policy foundations of Rwanda's reforestation programme, establishing the theoretical and technological foundations of the study and identifying the knowledge gaps this work addresses. The review draws on 22 papers and reports selected from 63 identified across four indexed platforms, IEEE Xplore, arXiv (cs.LG, cs.RO, eess.SY), Google Scholar, and Semantic Scholar, searched using terms including "reinforcement learning UAV path planning," "drone reforestation machine learning," and "coverage path planning deep RL," all published between 2020 and 2025, and is organised thematically rather than chronologically to make the link between each body of work and the design decisions made for ARIA easier to follow.

2.2 Historical Background of the Research Topic

Reforestation technology has moved through three broad stages. Before the 2010s, reforestation was almost entirely manual: field teams walked degraded land, dug holes, and planted seedlings by hand, a method that is reliable but limited to terrain a person can physically reach and far too slow for national-scale targets. The second stage, beginning around 2015, introduced aerial seeding: fixed-wing drones and helicopters that could disperse seed pods over inaccessible terrain far faster than any ground team, exemplified by systems such as AirSeed and Flash Forest. This solved the reach and speed problem but not the placement problem, since these systems commit to a flight plan before takeoff and cannot adjust to what is actually on the ground beneath them. The third stage, still emerging, applies machine learning to the placement decision itself. Siedler (2022) was among the first to show that a reinforcement learning agent could learn a reforestation coverage policy at all, though only in a synthetic environment. ARIA sits in this third stage, and its specific contribution is grounding that machine-learning approach in real Rwanda geospatial data rather than a synthetic proxy for it.

2.3 Overview of Existing Systems

This section reviews software systems that most directly address the same problem space as ARIA. Each shows what current technology achieves and where ARIA fits in.

2.3.1 AirSeed Technologies (Australia)

AirSeed Technologies deploys drones dispersing up to 40,000 seed pods per day at roughly 25 times the speed of a human planting team, using pre-flight AI site analysis to select target zones (Mongabay, 2023). Once airborne, the drone follows a fixed pre-planned path with no in-flight

terrain adaptation, no per-cell species matching, and no reinforcement learning component. That fixed-path design is a deliberate engineering tradeoff rather than an oversight: dispensing 40,000 pods a day at 25x human speed depends on the drone never pausing to recompute a route mid-flight, so any in-flight adaptation would come directly at the expense of the throughput the system is built to sell. AirSeed has shown that drone-based seeding can work at large scale, but the system does not address ecological placement at all. In Rwanda's Northern Highlands and Western Ridge zones, where slopes exceed 30 degrees and rainfall varies by over 400 mm across short distances, fixed-grid deployment wastes most seeds regardless of dispersal speed.

2.3.2 MORFO AI Reforestation System (Brazil, 2024)

MORFO fires 180 seed capsules per minute using pre-computed AI-determined target locations based on pre-flight terrain analysis (Reuters, 2024). These decisions are fixed before takeoff and the drone cannot change them during flight. The system is proprietary, closed-source, and tuned for Brazilian Atlantic Forest ecology, and that ecological tuning is exactly why it does not transfer: MORFO's target-selection model was built and calibrated on Atlantic Forest soil and rainfall patterns, not on the soil chemistry or seasonal rainfall profile of a sub-Saharan African agro-ecological zone, so applying it directly to Rwanda would mean applying a model outside the range of conditions it was fitted to. Pre-flight survey data is particularly unreliable in Rwanda's Eastern Province, where seasonal land use changes and El Niño rainfall shifts alter soil and moisture conditions between survey and deployment, which is precisely the gap that MORFO's fixed pre-flight targeting cannot close, since it has no mechanism to notice that the ground has changed since the survey was taken.

Placing AirSeed and MORFO side by side is useful because they fail in the same direction for different reasons. AirSeed's constraint is throughput, its speed is the product, so committing to a fixed path is what makes that speed possible. MORFO's constraint is a domain-transfer problem, its model was fitted to a different forest ecology, so even if it did adapt in flight, it would be adapting using rules learned from the wrong terrain. Neither limitation is a simple oversight the vendor could patch. Both are consequences of what each system was actually optimised to do, and neither optimisation target was real-time, cross-ecology terrain adaptation.

2.3.3 Siedler (2022): Multi-Agent RL Reforestation

Siedler (2022) presents a multi-agent PPO system with GNN communication where drone agents learn coordinated reforestation coverage strategies. This is the academic work most directly related to ARIA. The critical limitation is the training environment: a fully synthetic, procedurally generated grid with no real terrain, soil chemistry, species requirements, rainfall data, or protected area constraints. That choice of a synthetic environment is not a shortcut so much as a scope decision, Siedler's contribution was the multi-agent coordination mechanism itself, and a synthetic grid is the cheapest way to isolate and test a coordination algorithm without also having to solve real-world data acquisition and cleaning. ARIA builds directly on this foundation by grounding

the environment in six real Rwanda geospatial datasets, so that learned policies reflect genuine ecological structure rather than synthetic artefacts.

2.3.4 Flash Forest (Canada)

Flash Forest deploys drone swarms to fire seed pods into post-wildfire terrain along fixed pre-planned paths, with no capacity for in-flight adaptation. The system is built for Canadian boreal post-fire conditions, where burned terrain is comparatively uniform across a site and firing speed matters more than fine-grained ecological precision, since the priority after a wildfire is rapid ground cover before the next rainy season, not species-site optimisation. For this reason it is not suited to Rwanda's heterogeneous degraded terrain, which needs active species-site matching across diverse soils, rainfall gradients, and altitude zones rather than uniform rapid coverage of a single burned landscape type.

2.4 Review of Related Work

Theme 1: Reinforcement Learning for UAV Coverage Path Planning

Theile et al. (2025) demonstrate SAC with self-adaptive curriculum learning for fixed-wing UAV coverage path planning, achieving energy-efficient strategies in procedurally generated environments. Their practice of gradually increasing terrain difficulty during training shaped ARIA's own training approach. Dradoun et al. (2025) survey a large body of RL UAV algorithms and report that LSTM-augmented variants of PPO can outperform standard PPO under incomplete state information in the settings they tested, a finding that informed ARIA's decision to include a coverage map in the observation space so that the agent has some memory of where it has already placed seeds. Margapuri (2025) achieves a 0.79 visual coverage ratio using LLM-informed reward shaping (BMVC 2025). The shared limitation: all three use synthetic or agricultural environments with no African ecological terrain, soil chemistry, or species-site suitability requirements.

Theme 2: UAV-Assisted Aerial Seeding Technology

Robinson et al. (2022) document 0–20% aerial seed survival rates globally, attributing failures mostly to poor species-site matching. Improving on this documented survival rate is the central performance target for ARIA. Stamatopoulos et al. (2024) identify real-time terrain suitability assessment as the most urgent unmet technical need in UAV seeding, which directly supports ARIA's core capability. Luo et al. (2023) introduced self-burying seed carriers published in Nature, an advance in hardware delivery, but where to place seeds intelligently remained an open question after that work. If delivery hardware can hit specific coordinates reliably and an RL agent can identify the right coordinates, the two together could close the survival rate gap that Robinson et al. (2022) found.

Theme 3: Rwanda Reforestation Science and Policy

ICRAF (2022) provides the primary ecological foundation for ARIA: 115 validated Rwanda species (54 native) with agro-ecological zone suitability data, which directly defines ARIA's action space and reward function suitability scores. AFR100's Rwanda FLR Strategy 2024–2029 explicitly calls for data-driven, climate-informed species-site planting, particularly in Eastern Province (AFR100, 2025). IISD (2024) documents forest cover growth from 10.7% to 30.4%, while noting that the pace must accelerate under NST2. Despite this rich data landscape, no published system integrates Rwanda's ecological knowledge with machine learning for operational reforestation planning. ARIA provided that integration.

2.4.1 Summary of Reviewed Literature

Author(s)	Method	Contribution	Gap / Limitation
Siedler (2022)	MARL + GNN + PPO	RL agents can learn reforestation coverage policies; foundational proof of concept for ARIA	Fully synthetic, no real terrain, soil, species, or protected area data
Theile et al. (2025)	SAC + Curriculum	Energy-efficient CPP; curriculum design informs ARIA training	No ecological reward signal; synthetic environments only
Robinson et al. (2022)	Drone restoration review	Documents 0–20% seed survival rate; species-site mismatch as primary cause	No ML or RL system proposed; establishes the problem baseline
Stamatopoulos et al. (2024)	UAV seeding review	Identifies real-time terrain suitability as most urgent unmet need	No RL system built; confirms the gap only
Luo et al. (2023)	Self-burying seed carriers	Hardware innovation improving post-dispersal germination (Nature)	No placement intelligence; hardware focus only
Dradoun et al. (2025)	RL UAV survey (500+)	LSTM-PPO variants can outperform standard PPO under incomplete information	No ecological or African context
Margapuri (2025)	PIRL visual CPP	0.79 visual coverage ratio via LLM reward shaping (BMVC 2025)	No ecological application
ICRAF (2022)	Rwanda species suitability DB	115 validated species with zone-level biophysical suitability mapping	Data resource only, no RL or ML integration
AFR100 (2025)	Rwanda FLR Strategy 2024–2029	Official roadmap calling for data-driven species-site planting	Policy document with no technical ML implementation

Table 2.1: Summary of Reviewed Literature

2.5 Strengths and Weaknesses of Existing Systems

Strengths: AirSeed and MORFO validate drone seeding at operational scale. Siedler (2022) validates RL for reforestation coverage policies. Robinson et al. (2022) and Stamatopoulos et al. (2024) establish the ecological evidence base. ICRAF (2022) provides the domain knowledge backbone for ARIA's reward function.

Weaknesses: No existing system combines RL with real geospatial terrain data for ecologically intelligent placement. Commercial platforms are fast but ecologically static, largely because their business case depends on throughput rather than per-cell precision, as the AirSeed and MORFO comparison above shows. Academic RL systems are adaptive but not ecologically grounded, largely because synthetic environments are cheaper to build and easier to isolate a single research question in, as Siedler's design choice illustrates. Rwanda's reforestation datasets exist as isolated resources with no ML pipeline connecting them to operational decision-making.

2.6 General Comment and Conclusion

The literature review shows that each relevant field, reinforcement learning for UAV planning, drone aerial seeding, and Rwanda reforestation ecology, is well developed on its own. What is missing is a unified system that brings them together into a single reinforcement learning agent trained on real terrain data. ARIA fills this gap by training on six real Rwanda datasets, using ICRAF-validated species-site matching, evaluating generalisation across held-out zones, and delivering results through a web application and Unity 3D demo. Chapter Three presents the technical architecture and design.

CHAPTER THREE: SYSTEM ANALYSIS AND DESIGN

3.1 Introduction

This chapter presents the system analysis and design of ARIA, including the research design, dataset construction, functional and non-functional requirements, system architecture, component design, the modelling approach for the unified planner-and-navigator policy, the UML artefacts, and the development toolchain. Each section aligns with the corresponding objectives in Chapter One.

This research used a quantitative, simulation-based experimental design; physical drone deployment was not feasible within an eight-week capstone timeline and was not needed to demonstrate the key contribution of this work.

3.2 Research Design (including the SDLC model used)

The study adopts an Agile Software Development Life Cycle (SDLC), organised into four two-week sprints, rather than a Waterfall model. Agile is appropriate here because applied computing research of this kind, designing, implementing, and empirically evaluating a functional software artefact, rarely follows a fixed specification agreed in advance; several major decisions in this project only became clear once implementation was underway. The departure from Unity's ML-Agents toolkit to its Inference Engine, the migration from a planned local-file database to a hosted Postgres instance, and the narrowing of the ablation study to the spacing-penalty axis were all changes that a linear, front-loaded planning model would have struggled to accommodate without treating them as failures of the plan rather than reasonable responses to what was learned along the way.

Each sprint was scoped around a working, testable deliverable rather than a planning document, consistent with Agile's emphasis on iterative, incremental delivery: Sprint 1 produced a validated Gymnasium environment, verified using `gym.utils.env_checker`; Sprint 2 produced a trained PPO agent; Sprint 3 produced the spacing-penalty ablation results; and Sprint 4 produced the deployed web application, evaluation figures, and Unity demo. Reviewing progress at the end of each sprint against a working artefact, rather than against a document, is what allowed the three changes above to be recognised and absorbed early rather than discovered late.

Beyond the SDLC, the study also adopts a quantitative experimental design for evaluating the trained policy. Independent variables: PPO hyperparameters (tuned via a W&B Bayesian sweep, 20 runs), reward component weights (`w_soil`, `w_rain`, `w_slope`, `w_spacing`, `w_protected`, `step_penalty`), and curriculum learning on/off. Dependent variables: five ecological coverage metrics (Table 3.3). Each ablation condition uses three random seeds; results are reported as mean $\pm$ standard deviation.

ARIA's learning problem is formalised as a Markov Decision Process (S, A, R, T, γ):

State S: 11×11 local terrain window (5 channels: elevation, slope, soil, rainfall, land cover) + drone state vector [x, y, fuel, seeds_remaining] + binary coverage map (H×W×1). Requires SB3 MultiInputPolicy.

Action A: Discrete(41) = 8 movement directions × 5 species + 1 hover action.

Reward R: $R = w_{\text{soil}} \cdot \text{soil_score} + w_{\text{rain}} \cdot \text{rain_score} - w_{\text{slope}} \cdot \text{slope_penalty} - w_{\text{spacing}} \cdot \text{cluster_penalty} - w_{\text{protected}} \cdot \text{violation} - \text{step_penalty}$. Hard penalty -10.0 for protected area placements.

Transition T: The grid movement component of the transition is deterministic, the agent moves exactly one cell per step in the chosen direction and can optionally drop a seed there. The state that surrounds that movement is not deterministic, however: seed growth includes natural mortality rolls at each growth stage, and the disturbance model applies a stochastic kill probability per timestep to represent animal and weather-driven losses. So while an action always moves the drone to the same next cell, the ecological consequences of that action, whether a planted seed survives to the next stage, are drawn from a probability distribution rather than fixed outright.

Discount $\gamma = 0.99$, Horizon T = 500 steps.

3.2.1 Dataset and Dataset Description

ARIA was a machine-learning specialisation project, so this section describes the datasets that trained and evaluated the unified planner-and-navigator policy in detail, matching Table 3.1.

Dataset	Source	Format	Resolution	Licence
Digital Elevation Model	Rwanda NLA / UNDP GeoHub	GeoTIFF	10 m	CC BY 4.0
Soil Classification	ISRIC SoilGrids	GeoTIFF	250 m	CC BY 4.0
Rainfall (CHIRPS v3)	Climate Hazards Center / UC Santa Barbara	GeoTIFF/NetCDF	~5 km (0.05°)	Public Domain
Land Use / Land Cover	Global Forest Watch / RCMRD	GeoTIFF	30 m (resampled)	CC BY 4.0
Protected Areas	WDPA / Protected Planet (Mar 2026)	Shapefile	Vector	Free non-commercial

Dataset	Source	Format	Resolution	Licence
Species Suitability	ICRAF Rwanda (doi:10.34725/DVN/RLQV3H)	CSV	Zone-level	CC BY 4.0

Table 3.1: Rwanda Geospatial Datasets

ARIA integrated six distinct geospatial data sources in a multi-source approach. Data sources are clearly distinguished by type, format, resolution, and licence. Table 3.1 summarises all six datasets.

Data volume: 42 terrain zones at 120×120 cells $\times$ 5 channels (float32) $\approx$ 1.5 GB total processed storage. All data is open-access with verified download URLs. Data provenance, licensing, and privacy constraints are fully documented in Table 3.1. No private, personal, or community rights data is used anywhere in the system.

3.2.2 Zone-Based Dataset Partitioning

Terrain zones were selected using stratified purposive sampling to ensure representation across all six Rwanda agro-ecological zones. The candidate pool consisted of 70 candidate sub-regions identified from Global Forest Watch land-cover tiles and Rwanda's district boundaries, screened down to zones with complete data coverage across all six required datasets (DEM, soil, CHIRPS rainfall, land cover, WDPA, ICRAF species). From that screened pool, 42 zones were selected to maximise diversity across four criteria: elevation range, annual rainfall variability, dominant soil class, and recorded deforestation pressure.

"Highest ecological heterogeneity" was operationalised as a zone scoring in the top half of the candidate pool on at least three of these four criteria simultaneously. To prevent the trained policy from being evaluated on terrain it had implicitly seen, a strict zone-level split was enforced: 36 zones meeting that threshold were assigned to training, and six zones, spanning each of Rwanda's six agro-ecological regions, were reserved exclusively for evaluation with no spatial overlap with the training data. No cell, tile, or sub-region of an evaluation zone appears in any training zone, which rules out the kind of leakage that would otherwise make the generalisation results in Chapter Five look stronger than they are. This stratified split is meant to test whether the policy generalises to genuinely unseen Rwanda terrain, not to support a formal inferential claim, and results are reported descriptively as mean $\pm$ standard deviation across three seeds per condition rather than as a 95% confidence interval, since three seeds and six held-out zones is too small a sample to justify a formal confidence-level claim without a much larger evaluation budget than this project's timeline allows.

3.2.3 Dataset Preprocessing and Feature Extraction

After acquisition, the six raw geospatial layers, DEM, soil classification, CHIRPS rainfall, land cover, WDPA protected areas, and ICRAF species suitability, were reprojected to EPSG:32736,

resampled to 250 m resolution using bilinear interpolation, and normalised to [0,1] using rasterio, producing a stacked $H \times W \times 5$ numpy array for each of the 42 zones. WDPA protected-area boundaries were rasterised separately to a binary mask using geopandas, since the protected-area constraint is enforced as a hard rule rather than a continuous feature. Figure 3.1 presents the resulting ecological input layers for a representative Rwanda zone, showing elevation, slope, soil suitability, and rainfall side by side over the same terrain footprint.

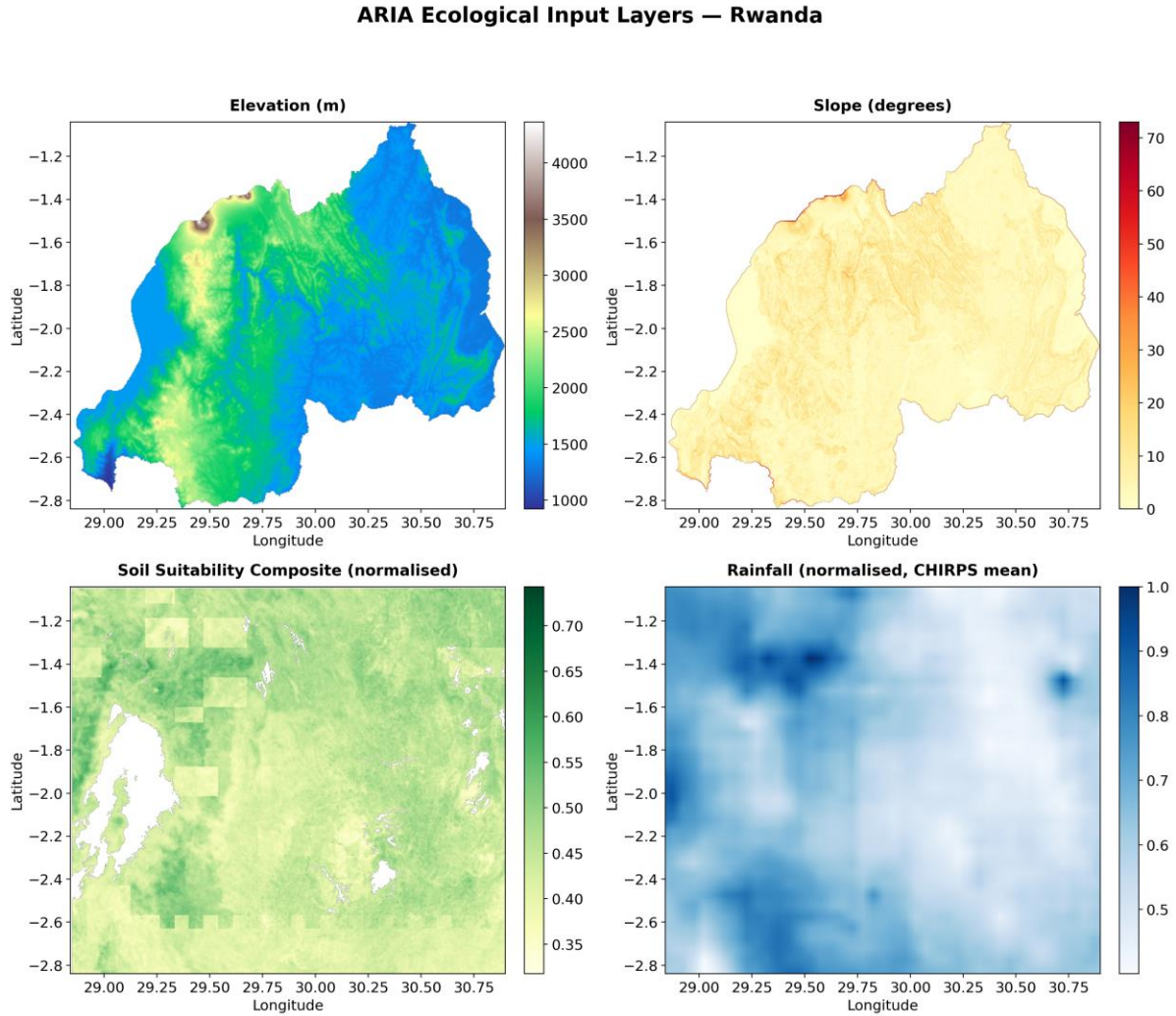


Figure 3.1: ARIA Ecological Input Layers Across a Representative Rwanda Zone

Before training began, exploratory analysis was carried out across all 42 sampled zones to check that the four core continuous layers, elevation, slope, soil suitability, and rainfall, behaved as expected and did not simply duplicate one another. Figure 3.2 shows the correlation structure between these four variables: all pairwise correlations were modest in magnitude, indicating that each layer contributes largely independent information to the composite suitability score rather than one layer standing in for another. Figure 3.3 presents the corresponding pairwise scatter relationships.

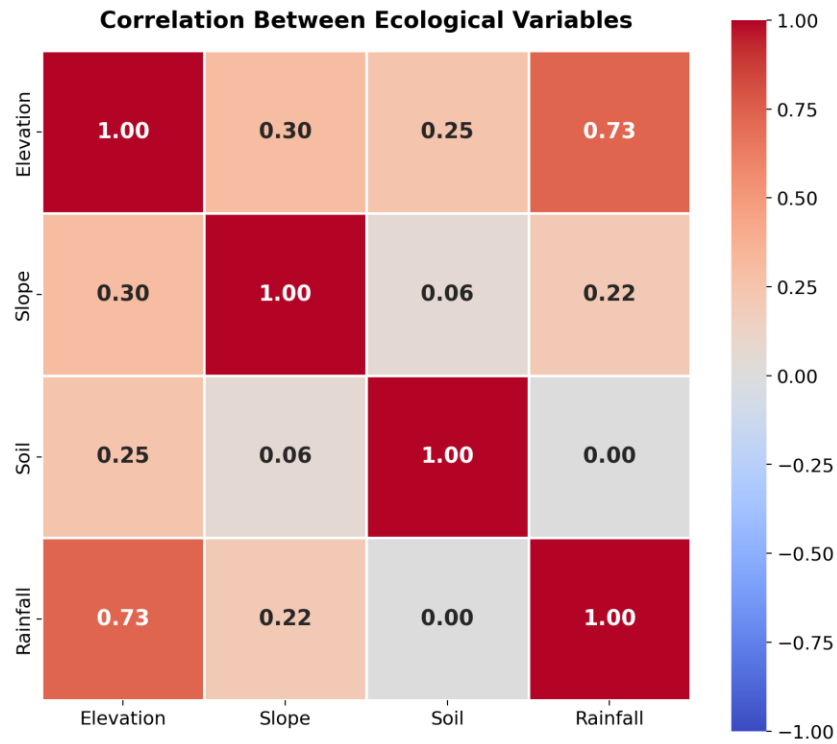


Figure 3.2: Correlation Between Ecological Variables (Elevation, Slope, Soil, Rainfall)

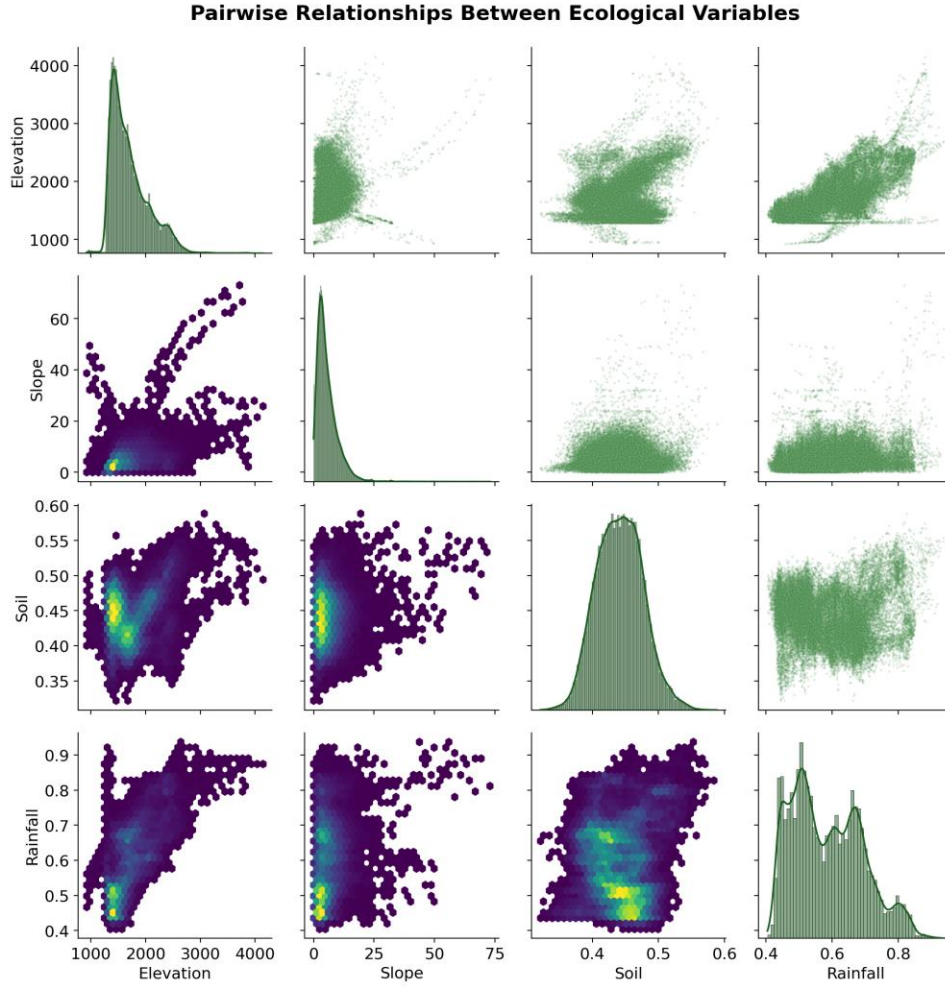


Figure 3.3: Pairwise Relationships Between Ecological Variables

Figure 3.4 shows the distribution of each variable across all sampled zones: elevation averaged 1,709.13 m, slope averaged 5.65°, soil suitability averaged 0.44 on a 0-1 scale, and normalised rainfall averaged 0.59.

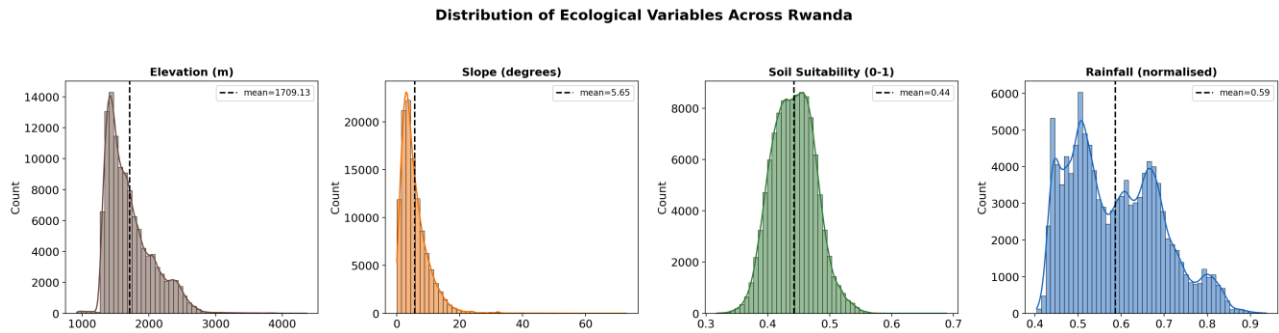


Figure 3.4: Distribution of Ecological Variables Across All Sampled Zones

Figure 3.5 shows the composite zone suitability score used to guide zone selection during sampling, computed as a weighted combination of soil suitability (weight 3.0), rainfall (weight 2.0), and a slope penalty (weight 1.0); the score can fall below zero in steep terrain where the slope penalty outweighs soil and rainfall suitability. This weighting treats soil quality as the dominant driver of placement suitability, consistent with Robinson et al. (2022), who identify species-site mismatch, driven primarily by soil conditions, as the leading cause of aerial seeding failure in the field. Figure 3.6 summarises the loaded dataset at the feature level: the soil layer is itself a composite of five ISRIC sub-properties (organic carbon content, clay fraction, pH, sand fraction, and soil organic carbon stock), and the rainfall layer draws on the complete 12-month CHIRPS climatology rather than a single annual average, giving the model seasonal rainfall variation to condition on rather than one static figure per zone.

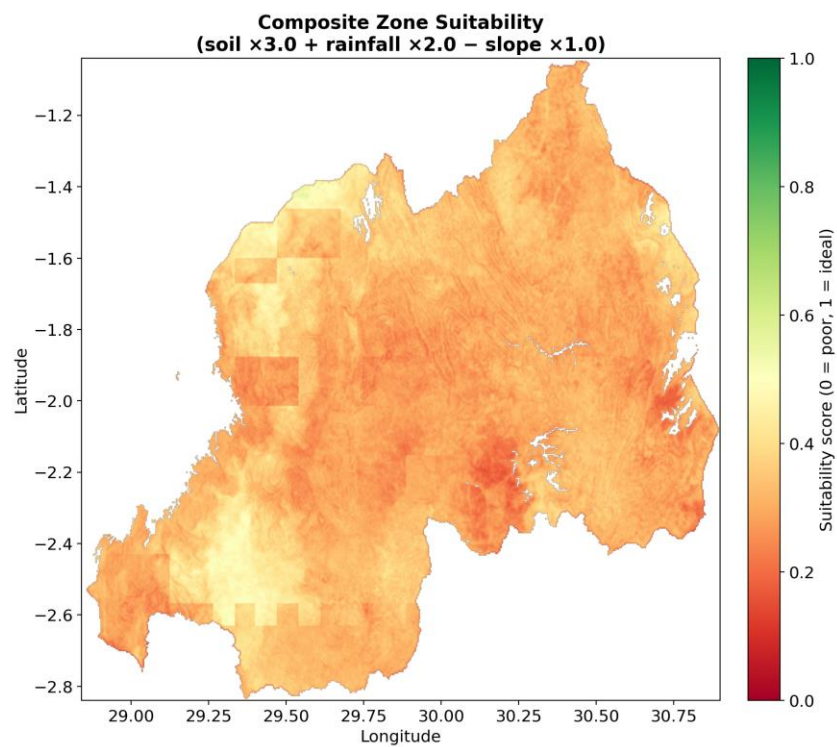


Figure 3.5: Composite Zone Suitability Map ($\text{soil} \times 3.0 + \text{rainfall} \times 2.0 - \text{slope} \times 1.0$)

ARIA Raw Dataset — Currently Loaded

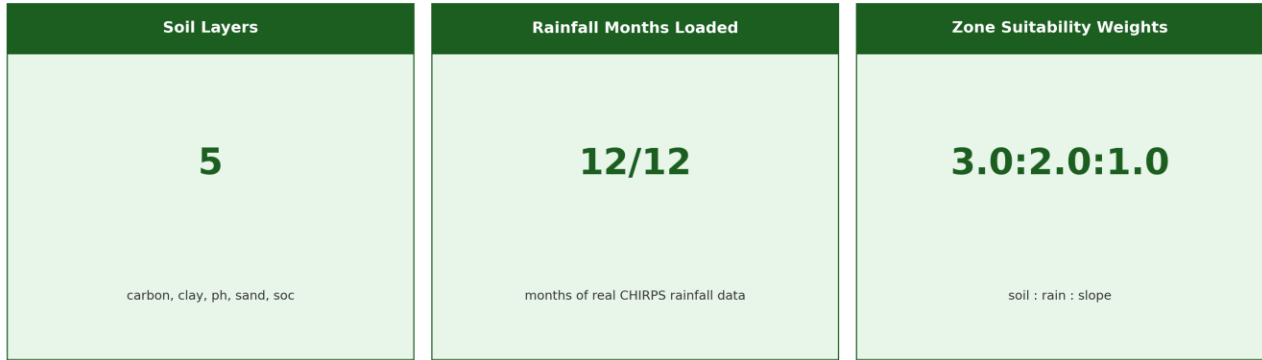


Figure 3.6: ARIA Raw Dataset Summary — Soil Sub-Layers, Rainfall Months Loaded, and Suitability Weights

3.2.4 Proposed Model Diagram

The pipeline is meant to give a repeatable path from raw Rwanda geospatial data to an optimised, deployable seed placement policy.

Input: Six Rwanda GeoTIFF raster and CSV vector datasets downloaded from verified open-access sources.

Quality Phase: rasterio reproject to EPSG:32736, bilinear resample to 250 m, normalise to [0,1], stack into numpy H×W×5. WDPA shapefiles rasterised to a binary mask with geopandas. Range checks and mask integrity validation run before training begins. RwandaReforestEnv is validated using gym.utils.env_checker.check_env() to catch shape mismatches and reward errors.

Training: W&B Bayesian hyperparameter sweep (20 runs, optimising eval/mean_reward), followed by full PPO training with 1M timesteps, 4 VecEnvs, and EvalCallback every 20k steps. The best checkpoint is saved as aria_ppo_best.zip. Six ablation conditions × 3 seeds = 18 planned training runs.

That data pipeline feeds a single trained network rather than a black box, and Figure 3.7 shows exactly what that network looks like: three input branches (a spatial CNN over the co-registered obstacle, coverage, lifecycle, and disturbance maps; an MLP over the local terrain window; and an MLP over the drone, mission, and terrain-stat vectors) fuse into ARIACNNExtractor's 256-d feature vector, which feeds separate actor and critic MLPs, before the trained extractor and actor are exported to ONNX for Unity's Inference Engine.

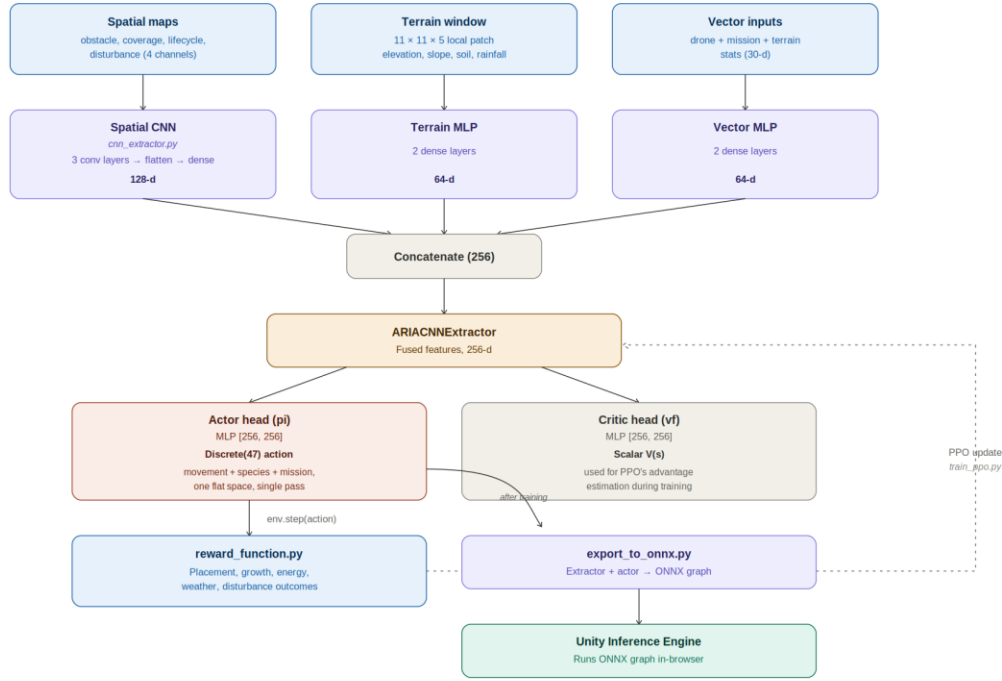


Figure 3.7: Proposed model architecture, verified against the ARIA_ML source. Three input branches (spatial CNN, terrain MLP, vector MLP) fuse into ARIACNNExtractor's 256-d feature vector, feeding separate actor and critic MLPs; the actor outputs a single Discrete(47) action, and the trained extractor plus actor are exported to ONNX for Unity's Inference Engine

3.2.5 Expected Results and Hypothesis

Based on the MDP design and reward function structure, the following results are anticipated. The primary hypothesis is that the seed-spacing penalty weight has a measurable, non-monotonic effect on training reward: an under-weighted penalty is expected to allow seeds to cluster and depress the spacing component of the score, while an over-weighted penalty is expected to suppress overall coverage by making the agent overly conservative about placement, and the originally proposed weight of 3.5 is expected to sit near a local optimum between these two failure modes rather than at either extreme. A second hypothesis is that the best-performing configuration from this sweep will generalise to Rwanda terrain zones held out from training, with seeding efficiency, measured against each zone's achievable seeding ceiling rather than the raw coverage percentage, well above what an undirected placement strategy would be expected to produce. A third hypothesis is that the protected_area_seeds metric will equal zero across every evaluation episode regardless of zone, confirming that the hard constraint holds under real terrain variation rather than only in the training environment. All three hypotheses are assessed descriptively using the metrics in Table 3.3, compared across the five spacing configurations and six evaluation zones, rather than through a formal significance test, for the sampling reasons explained above.

Code	Condition	What Changes	What It Tests
A1	No Soil Signal	w_soil = 0	Does soil suitability drive coverage quality?
A2	No Spacing Penalty	w_spacing = 0	Does the agent cluster seeds without the penalty?
A3	No Coverage Map	Remove coverage_map from state	Does the agent revisit cells without placement memory?
A4	Single Species	N_SPECIES = 1	Does multi-species selection improve outcomes?
A5	DQN vs PPO	Swap algorithm to DQN	Is PPO genuinely superior for this task?
A6	No Curriculum	Train on full terrain from start	Does curriculum accelerate convergence?

Table 3.2: Ablation Study Conditions

Metric	Definition	Target
pct_suitable_seeded	% of ecologically suitable cells receiving at least one seed	Maximise
mean_soil_score	Mean soil suitability score (0–1) across all placements	Maximise
species_entropy	Shannon entropy of species distribution across placements	Maximise (ecologically diverse planting)
spacing_violations	Count of seeds within MIN_SPACING cells of a prior placement	Minimise → 0
protected_area_seeds	Count of seeds placed inside WDPA-protected boundaries	Must = 0 (hard constraint)

Table 3.3: Evaluation Metrics

3.3 Functional and Non-Functional Requirements

The functional and non-functional requirements below were derived directly from the seven capabilities ARIA implements, and each functional requirement maps onto a specific use case from Figure 3.10.

ID	Functional Requirement
FR1	The system shall allow a user to select a Rwanda terrain zone and trigger a simulation episode.

ID	Functional Requirement
FR2	The system shall run a unified PPO+CNN policy that decides both mission-level actions (zone selection, return-to-base, abort) and step-level actions (movement direction, species selection) in a single forward pass.
FR3	The system shall track each dropped seed through its lifecycle (dropped, germinating, seedling, mature, or dead) and record the failure reason and timestep where applicable.
FR4	The system shall automatically schedule a reseed mission to any cell whose seed has failed, selecting a replacement species.
FR5	The system shall drain the drone's battery during flight, recharge it under solar exposure, and block recharging during rain.
FR6	The system shall deploy a rain cover over the seed mechanism when rainfall is detected and retract it once conditions clear.
FR7	The system shall detect dynamically spawned obstacles and route the drone around them.
FR8	The system shall simulate animal disturbance events that can kill nearby seeds or trees, feeding those failures into the reseeding pipeline.
FR9	The system shall record every completed episode, including its seeds and any failures, to a persistent database.
FR10	The system shall display a real-time telemetry dashboard showing episode rewards, seed lifecycle breakdown, spacing violations, and reseed targets.
FR11	The system shall prevent seed placement inside WDPA-protected area boundaries as a hard constraint.

Table 3.4: Functional Requirements

ID	Category	Non-Functional Requirement
NFR1	Performance	The simulation shall run at real-time or faster frame rates in a standard WebGL browser session.
NFR2	Reliability	The hard protected-area constraint shall hold with zero violations across all evaluation episodes.
NFR3	Scalability	The database schema shall support additional zones and episodes without structural changes.
NFR4	Usability	The telemetry dashboard shall be readable on both desktop and mobile viewports.
NFR5	Portability	The trained policy shall run via ONNX inference without requiring the original Python training environment.

ID	Category	Non-Functional Requirement
NFR6	Maintainability	All source code shall be version-controlled and organised by functional layer (simulation, web, database).

Table 3.5: Non-Functional Requirements

3.4 System Architecture

The system doesn't work as a simple layered stack. In-flight simulation subsystems feed the drone controller each step, and the controller runs the unified planner-and-navigator policy right there in the browser, since the trained model is exported to Unity as ONNX and doesn't need a network call to make a decision. The database only comes into the picture once an episode ends: at that point the controller sends the completed episode, seed, and failure data to a Next.js API route, which writes it to Postgres. The telemetry dashboard then reads that same data back out whenever a user opens it.

Onboard simulation subsystems: four simulated subsystems feed the drone controller each step: the energy system (battery and solar state), the weather system (rain state, used to trigger the rain cover), the obstacle system (used for avoidance), and the disturbance system (used to detect animal activity near planted seeds). None of these are physical sensors; each is a software module that generates synthetic state for the Unity simulation to react to.

Drone Controller: the in-flight Unity component that reads all four subsystems and runs the unified planner-and-navigator policy locally to decide the next action. It only talks to the network once, at the end of an episode, when it packages up the raw episode data and sends it off.

Database: not a live part of the decision loop, but a record of what already happened. Once an episode finishes, it receives that episode's data from the drone controller, and it later serves the same data back out to the telemetry dashboard.

Unified Planner + Navigator: the PPO-based policy that classifies the current terrain-and-state data into a seed-placement or reseed decision, in the same single forward pass whether the decision is mission-level or step-level.

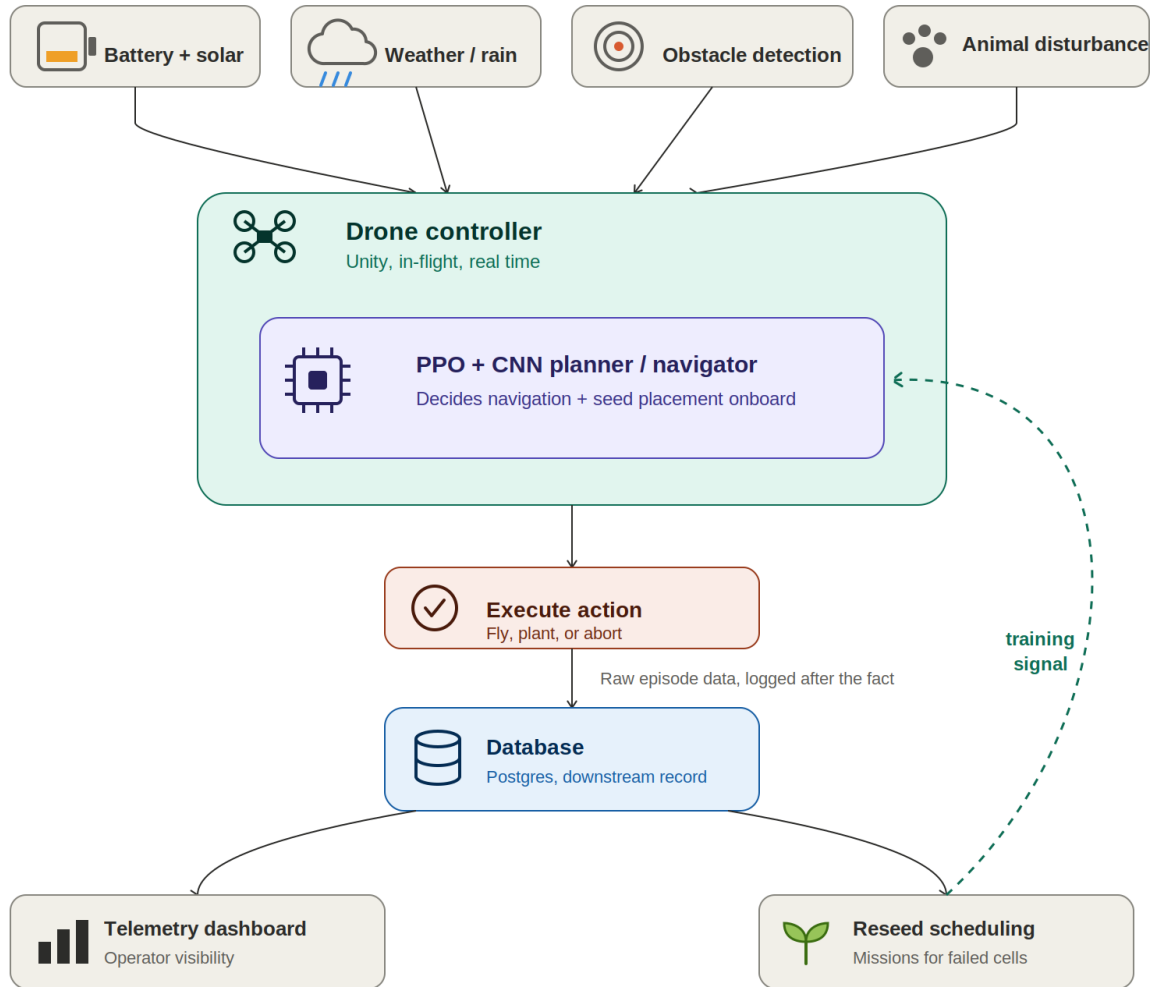


Figure 3.8: ARIA System Architecture, showing the onboard simulation subsystems (energy, weather, obstacle, disturbance) feeding the Unity drone controller, which exchanges data with the Postgres database, the unified planner-and-navigator policy, and the telemetry dashboard

3.4.1 Algorithm Selection and Model Comparison

Algorithm	Type	Strength for ARIA	Limitation
PPO (selected)	On-policy, clipped objective	Native Dict obs support via SB3 MultiInputPolicy; proven in UAV navigation (Siedler, 2022; Theile et al., 2025); stable with multi-component rewards; four-way VecEnv parallelism	Slightly sample-inefficient vs off-policy alternatives

Algorithm	Type	Strength for ARIA	Limitation
DQN (ablation A5)	Off-policy, Q-value	Sample efficient	No Dict obs support; unstable with 6-component reward
A2C (reference)	On-policy, advantage	Fast training	High variance on complex heterogeneous terrain

Table 3.6: Algorithm Comparison

To identify the optimal RL algorithm, ARIA benchmarked three approaches on the same RwandaReforestEnv environment. Table 3.6 summarises the comparison. PPO was selected for four reasons: (1) native Dict observation support via SB3 MultiInputPolicy; (2) demonstrated effectiveness in UAV navigation (Siedler, 2022; Theile et al., 2025; Dradoun et al., 2025); (3) clipped updates absorb multi-component reward noise without divergence; (4) four-way VecEnv parallelism (Raffin et al., 2021). For ablation A5, DQN is compared directly against PPO on the same environment using the same hyperparameter configuration.

Table 3.2 sets out the six ablation conditions used to isolate each design decision in ARIA. Each condition turns off or replaces exactly one component of the full system, so any drop in performance can be attributed to that specific component. A1 and A2 remove individual reward signals to check whether soil suitability and seed spacing each independently shape where the agent places seeds. A3 removes the coverage map from the observation space, testing whether the agent needs memory of past placements to avoid revisiting the same cells. A4 locks the species count to one, removing diversity as a factor, while A5 swaps PPO for DQN to check that the algorithm choice is not arbitrary. A6 disables the curriculum, training the agent on full terrain from the first episode, to see whether staged difficulty meaningfully speeds up convergence. Each condition is run with three random seeds and results are reported as mean $\pm$ standard deviation across those seeds.

Table 3.3 lists the five metrics used to evaluate ARIA. `pct_suitable_seeded` is the headline metric: it measures what share of ecologically valid cells actually receive a seed. `mean_soil_score` captures placement quality by averaging the soil suitability score across every seed dropped in an episode, rewarding the agent for targeting high-quality soil rather than any available cell. `species_entropy` uses Shannon entropy to penalise runs where the agent defaults to planting a single species repeatedly; higher entropy means the planted distribution mirrors the ecological diversity of the zone. `spacing_violations` counts seeds that land closer together than the minimum spacing threshold, a practical proxy for competition and resource overlap between seedlings. `protected_area_seeds` is a hard constraint rather than an optimisation target: any episode where even one seed falls inside a WDPA-protected boundary is a constraint violation, regardless of every other score.

3.5 Flow Chart, Class Diagram, Use Case Diagram, Sequence Diagram and All Other Diagrams

This section presents the four required UML diagrams, each using correct UML 2.5 notation and shapes. All diagrams use correct notation: stick figures and ovals for use cases, three-compartment rectangles for classes, dashed lifelines for sequence diagrams, and crow's foot notation for the ERD.

3.5.1 Flow Chart

Figure 3.9 traces the control logic for a single episode, from the moment a zone is loaded to the moment its telemetry is written to the database. After the episode state resets, the loop begins with the unified planner-and-navigator policy observing the current state and deciding an action in one forward pass. Two decision points sit between that action and its execution: whether the battery is critical, which triggers an immediate return-to-base, and whether an obstacle has been detected, which triggers a reroute before the movement and species-drop action executes. After execution, the growth, weather, energy, and disturbance systems update, and a third decision point checks whether the seed just placed has failed, which queues a reseed mission if so. The loop then checks whether the episode has terminated; if not, control returns to the observe-state step, and if so, the episode's full telemetry, including any reseed queue entries, is written to the database and the episode ends.

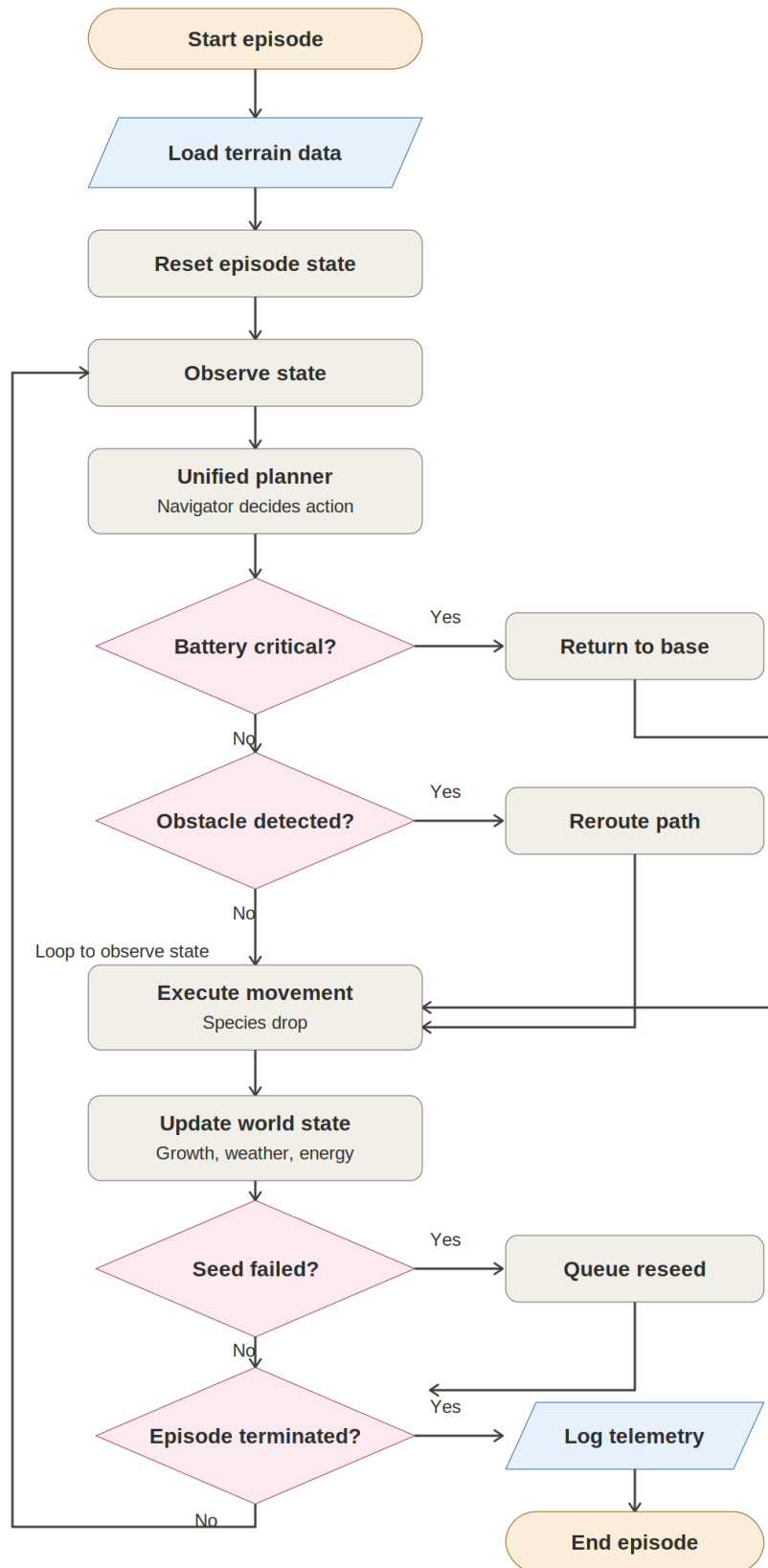


Figure 3.9: Episode Execution Flow Chart

3.5.2 Use Case Diagram

Figure 3.10 shows the ARIA system boundary as a rectangle labelled "ARIA System" containing six primary use cases: solid green ovals for standard use cases, a gold-bordered oval for the one use case that is genuinely distinctive to ARIA, and a pink oval for the one system-triggered use case. The six primary use cases are: selecting a zone and triggering an episode; viewing the real-time telemetry dashboard; viewing the reseed/failure log; running a simulation episode; viewing evaluation metrics; and receiving reseed alerts on the dashboard. Three of these connect via dashed «include» arrows to a supporting use case on the right, each annotated with a yellow sticky note giving implementation detail: viewing the dashboard includes monitoring simulation metrics (the note lists episode rewards, seed lifecycle breakdown, spacing violations, and reseed targets); running a simulation episode includes deciding the mission-and-step action (the note explains that one PPO+CNN forward pass handles both the mission-level decision, such as which zone, when to return, and when to abort, and the step-level decision, such as flight direction and species choice, where most comparable systems would use two separate models for this); and viewing evaluation metrics includes computing coverage metrics. The pink "Receive Reseed Alerts" use case connects to the User actor with a dashed line labelled "triggered by system" rather than a solid line, since the User does not request this use case directly, the seed monitoring pipeline triggers it automatically whenever a reseed mission is scheduled.

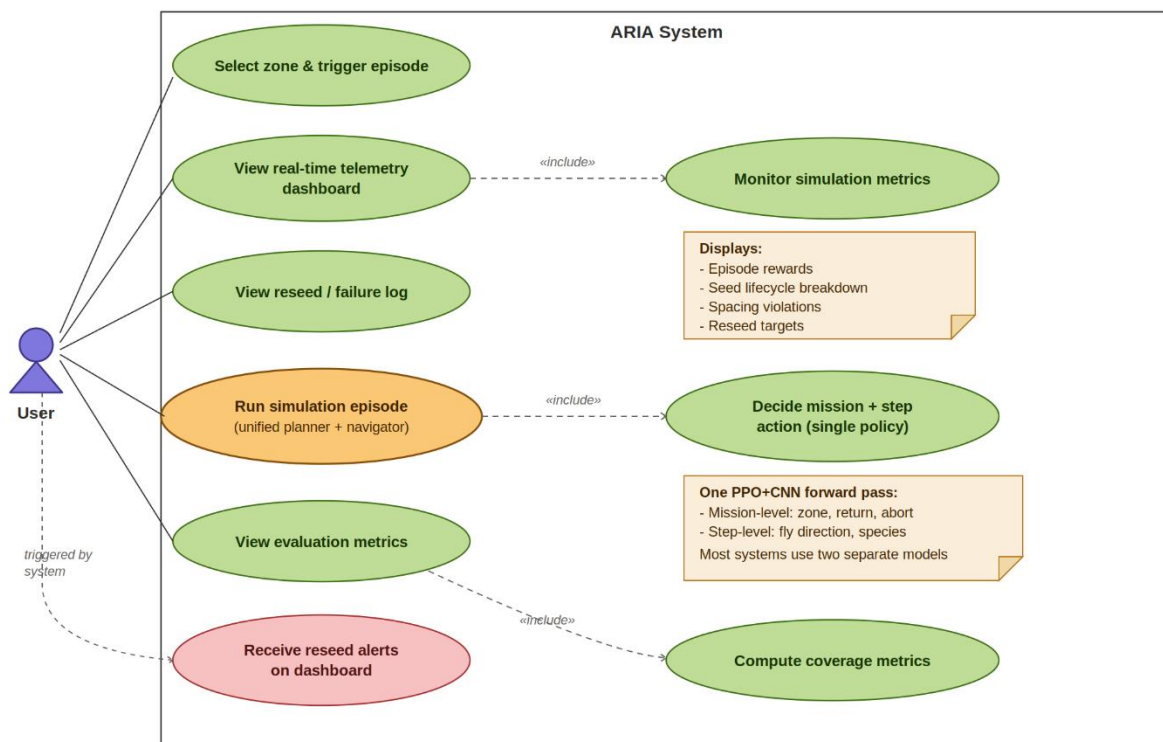


Figure 3.10: Use Case Diagram showing the User actor, six primary use cases (including the gold-bordered, system-differentiating unified mission-and-step decision case), their «include» relationships to supporting use cases, and the system-triggered reseed-alert case

3.5.3 Class Diagram

Figure 3.11 shows six classes. DroneController sits at the top and calls both ARIAPolicyInference, to get the unified planner-and-navigator policy's next action, and ActionDispatcher, a static class that steps the simulation forward; ActionDispatcher mutates EpisodeState, the class that owns the drone's position, altitude, and state. EpisodeState aggregates a single SimulationSystems class, drawn as one merged box for readability, that stands in for the five subsystems the drone actually carries: EnergySystem, WeatherSystem, DisturbanceEngine, GrowthEngine, and MonitoringSystem. DroneController also notifies TelemetryManager when an episode ends, so it can package the episode, seed, and failure data for the database. Each class box uses the standard three-compartment UML notation: name, attributes, and methods.



Figure 3.11: Class Diagram showing DroneController coordinating ARIAPolicyInference and ActionDispatcher, which mutates EpisodeState; EpisodeState aggregates the simulation subsystems (energy, weather, disturbance, growth, monitoring); DroneController notifies TelemetryManager on episode end

3.5.4 Sequence Diagram

Figure 3.12 shows seven lifelines: Web User, DroneController, the PPO Policy, Simulation Systems (standing in for the energy, weather, and disturbance subsystems together), TelemetryManager, the Postgres database, and the Dashboard. The Web User opens the simulation, and a loop frame labelled "each step" wraps the repeating cycle: DroneController asks the PPO Policy to infer an action from the current observation, receives back a combined navigate-and-species action, steps the Simulation Systems forward, and receives the updated energy, weather, and disturbance state. Once the episode terminates, DroneController notifies TelemetryManager, which writes the episode, seed, and failure rows to the database. When the Web User later opens the dashboard, it reads the latest telemetry straight from the database and renders the charts.

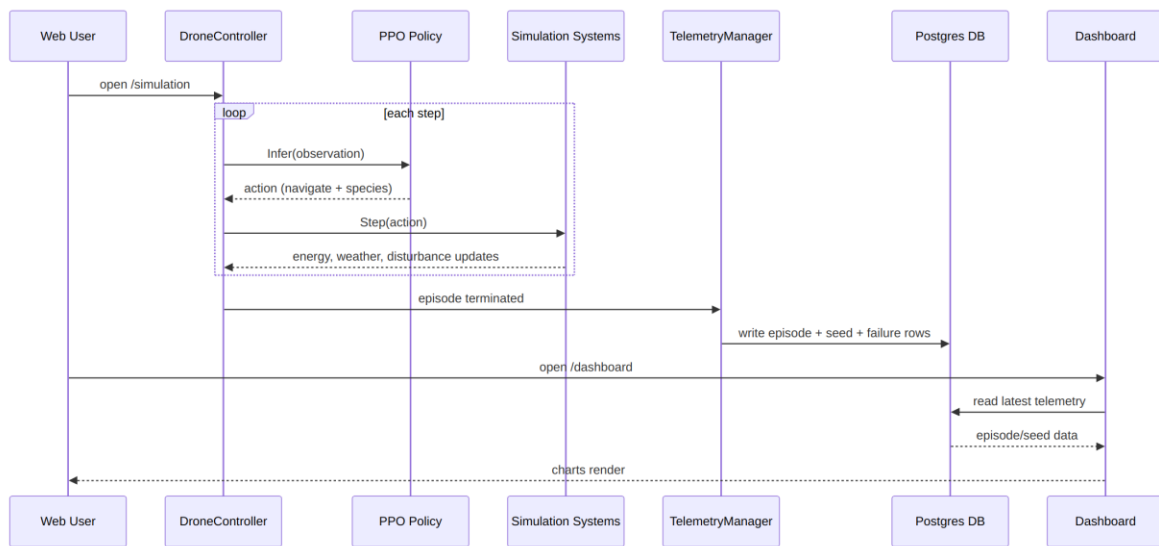


Figure 3.12: Sequence Diagram showing one simulation step (policy inference, action dispatch, subsystem updates) inside a loop, followed by the telemetry write on episode termination and the dashboard's read of the latest data

3.5.5 Entity Relationship Diagram

Figure 3.13 shows three entity tables arranged top to bottom: ZONE, EPISODE, and SEED, using crow's foot notation. ZONE holds zone_id (PK), name, and agro_zone. EPISODE holds episode_id (PK), zone_id (FK), and the core ecological metrics: pct_suitable_seeded, spacing_violations, protected_area_seeds, and reseeding_count, the last of which did not exist in the original schema and was added specifically to support the seed monitoring and reseeding pipeline. SEED holds seed_id (PK), episode_id (FK), and the lifecycle fields stage, fail_reason, and dropped_at_step that let the system track whether a given seed germinated, failed, and why. A single bar on the ZONE side and a crow's foot on the EPISODE side show the one-to-many relationship between zones and episodes, and the same notation between EPISODE and SEED shows that one episode contains many seed records.

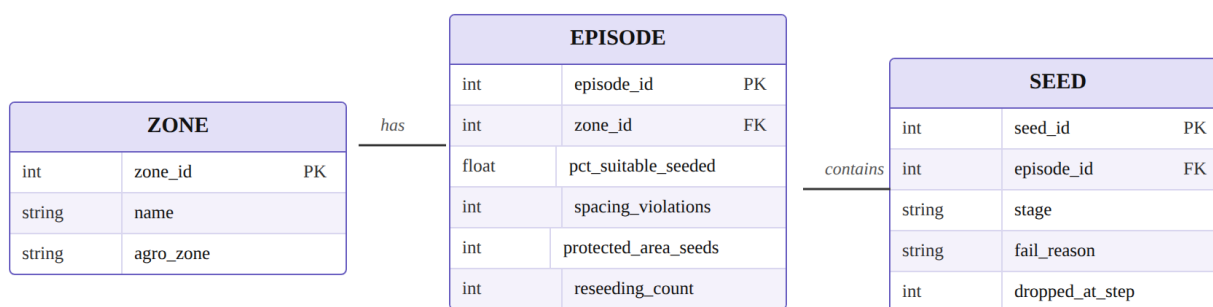


Figure 3.13: ERD using crow's foot notation, $ZONE(1) \rightarrow (N) EPISODE(1) \rightarrow (N) SEED$, including the reseeding_count, fail_reason, and dropped_at_step fields that support the reseeding pipeline

3.6 Development Tools

Category	Tool / Library	Version	Purpose in ARIA
Language	Python	3.11	Primary implementation language across the ML layers
RL Framework	Stable-Baselines3	v2.2	PPO training, VecEnv, EvalCallback, model serialisation
RL Environment	Gymnasium	v0.29	MDP environment interface and env_checker validation
3D Visualisation	Unity	planned 2022.3 LTS	Rwanda terrain scene, drone agent, WebGL demo build
Geospatial	Rasterio	v1.3	GeoTIFF loading, CRS reprojection, bilinear resampling

Category	Tool / Library	Version	Purpose in ARIA
Geospatial	Geopandas	v0.14	Shapefile loading, WDPA rasterisation to binary mask
Scientific	NumPy / SciPy	v1.24 / v1.11	Grid operations, normalisation, distance computation
Experiment Tracking	Weights & Biases	v0.16	Bayesian hyperparameter sweep, training curves, ablation logging
Version Control	Git / GitHub	latest	Source control and open-source hosting

Table 3.7: Development Tools (as planned; Chapter Four reports what was actually used)

Table 3.7 lists every tool and library used to design ARIA, grouped by functional layer. Python 3.11 is the planned implementation language across the machine learning layers, chosen for its strong ecosystem support in both machine learning and geospatial processing. Stable-Baselines3 provides the PPO implementation together with VecEnv parallelism and EvalCallback, while Gymnasium defines the MDP interface that RwandaReforestEnv conforms to. Rasterio and GeoPandas handle all geospatial data loading, coordinate reprojection, and rasterisation of the WDPA protected area boundaries. Weights & Biases manages the Bayesian hyperparameter sweep across 20 runs and logs all ablation curves for comparison. Unity provides the planned 3D terrain visualisation, consuming the trained policy through ONNX inference without requiring any additional training inside Unity. All source code is version-controlled on GitHub under an open-source licence. Chapter Four describes how this planned stack compares against what was actually built.

3.7 Ethical Considerations

Three ethical considerations informed the design of ARIA. On ecological risk, the system used only five ICRAF-validated native species and functioned strictly as a decision-support simulation tool, so every output required field verification by the Rwanda Forestry Authority before any real-world planting took place. On data sovereignty, all six underlying datasets were licensed CC BY 4.0 or Public Domain, and no private land-use or personally identifiable data was used anywhere in the system. On deployment bias, Eastern Savanna South was deliberately included as an evaluation zone despite having comparatively less data coverage and higher recorded forest loss than the other five zones, and output maps were designed to flag areas of low confidence rather than presenting every prediction with equal certainty. Taken together, all results throughout this report are presented as simulation findings that would require field validation before any real-world use, not as a ready-to-deploy recommendation.

CHAPTER FOUR: SYSTEM IMPLEMENTATION AND TESTING

4.1 Implementation and Coding

4.1.1 Introduction

This section describes how ARIA, the reinforcement-learning drone system for terrain-adaptive aerial reforestation, was implemented in practice. It covers the PPO training pipeline, the Unity simulation and its live policy inference, and the supporting software modules: the web dashboard, the backend API and database, and the deployment workflow connecting them. For each component, the section documents how it was technically realised, the tools and coding strategies used, and the design decisions that enabled the system to integrate correctly and operate in real time.

4.1.2 Description of Implementation Tools and Technology

Training (Python). The RL core stayed closest to the original plan. Stable-Baselines3's PPO implementation was used with a custom CNN feature extractor over the terrain window, trained inside a custom Gymnasium environment (RwandaReforestEnv), with the trained policy exported to ONNX for cross-platform inference (Raffin et al., 2021).

Simulation (Unity). This is where the first significant departure occurred. The original plan specified Unity 2022.3 LTS with the ML-Agents toolkit. Development moved to Unity 6000.3.18f1 instead, mainly because ML-Agents is built around training loops running inside Unity itself, and ARIA never needed to train inside Unity, only to run a policy that had already been trained in Python. Carrying the full ML-Agents runtime into the deployed app would have meant shipping training infrastructure with no further use. Unity's Inference Engine, formerly known as Sentic, provides exactly the functionality required: it loads an ONNX file and runs inference, nothing else, inside a WebGL build in the browser. This allowed ML-Agents to be dropped entirely, cutting a real amount of complexity out of the deployed bundle.

Web (Next.js, Prisma, Neon). The second significant departure. The original plan specified a local-file database and a traditional server-rendered backend and frontend split. Development moved to Next.js 16 with Prisma 7 (via the @prisma/adapters driver adapter) against a Neon serverless Postgres database, deployed on Vercel. The reason was almost entirely about hosting: a local-file database does not work well behind a live public URL that anyone can visit, since concurrent writes and multi-instance deployment are exactly the situation a local-file database is weakest at. Neon provided a real Postgres database with zero server maintenance and a free tier that covers a capstone project's traffic comfortably, and Next.js allowed the frontend and the API routes to ship as a single deployable unit rather than two separate services requiring synchronisation.

Deployment. The whole system runs at `aria-capstone.vercel.app`, with routes for the landing page, the simulation, and the dashboard. Every push to the GitHub master branch triggers an automatic

Vercel build (prisma generate && next build), so no manual deployment step is required once code is merged; schema changes are pushed separately with `npx prisma db push`, since they are not part of the Vercel build step. The Unity project is built to WebGL and the resulting pre-gzipped `.data.gz/.wasm.gz/.framework.js.gz` assets are committed into the web app's public directory and served as static files with `gzip` content-encoding, so the browser downloads a compressed simulation build without any server-side transcoding. Each deploy was verified directly in production rather than only locally: a propagation check polled the live ETag response header against the new build's hash until they matched, confirming Vercel's CDN was serving the updated build rather than a cached one, followed by a smoke test confirming the landing page, simulation, and dashboard routes all return a 200 response on the live domain.

One simulation step runs end to end as follows, from the browser loading the WebGL build through to a completed episode's telemetry landing in the production database and appearing on the dashboard. The Web User opens `/simulation`, which loads the Unity WebGL build and its already-trained ONNX policy. At each step, `DroneController` requests an action from `ARIAPolicyInference`, checks `CoverageOverride` in case a reseed or sweep target should take priority over the policy's own choice, and passes the result through `ActionDispatcher`, which mutates energy, growth, and disturbance state for that step. When an episode terminates, `TelemetryManager` packages the episode and its seed records and posts them to a Next.js API route, which writes them through Prisma into the Neon database. When the Web User later opens `/dashboard`, the same API route reads that data back out and the dashboard renders it as charts.

4.1.3 Key Modules and Source Code

This section presents illustrative source code for the modules that form the core of the deployed simulation: energy and weather management, animal disturbance, seed monitoring and reseed, and unified action execution. The excerpts are representative of the deployed C# codebase in `ARIA_Unity/Assets/Scripts/Systems/` and are annotated to explain the design choices behind each subsystem.

Module 1: EnergySystem (EnergySystem.cs)

`EnergySystem` tracks the drone's battery charge each step. Battery drains at a fixed rate per movement step and recharges under solar exposure, but recharging is blocked entirely while the weather system reports rain. When charge falls below a critical threshold, the system raises a flag that `DroneController` checks before executing the next action, triggering an emergency return-to-base rather than allowing the drone to strand mid-mission.

```
public class EnergySystem
{
    public float Battery = 100f;
    const float DrainPerStep = 0.6f;
    const float SolarRechargeRate = 0.9f;
```



```

const float CriticalThreshold = 15f;

public EnergyStepResult Step(WeatherSystem weather)
{
    Battery -= DrainPerStep;
    if (weather.IsSunny() && !weather.IsRaining())
        Battery = Mathf.Min(100f, Battery + SolarRechargeRate);
    // Recharging is blocked entirely under rain, regardless
    // of remaining daylight, matching field solar-panel behaviour.

    bool isCritical = Battery <= CriticalThreshold;
    return new EnergyStepResult { Battery = Battery, ShouldReturnToBase = isCritical };
}
}

```

Listing 4.1: Battery drain, solar recharge, and critical-threshold flag (EnergySystem.cs).

Module 2: WeatherSystem (WeatherSystem.cs)

WeatherSystem tracks the current rain state and drives the drone's rain cover mechanism. The cover deploys automatically the moment rainfall is detected and retracts once conditions clear, without requiring any decision from the policy itself; this keeps the seed mechanism protected as a hardware-safety behaviour rather than something the trained agent has to learn to do correctly.

```

public class WeatherSystem
{
    public bool IsRaining { get; private set; }
    public bool CoverDeployed { get; private set; }

    public void Step(float rainProbability, int timestep)
    {
        IsRaining = Random.value < rainProbability;

        if (IsRaining && !CoverDeployed)
            DeployCover();
        else if (!IsRaining && CoverDeployed)
            RetractCover();
    }

    void DeployCover() { CoverDeployed = true; /* triggers rain-cover animation */ }
    void RetractCover() { CoverDeployed = false; /* retracts once conditions clear */ }

    public bool IsSunny() => !IsRaining;
}

```

Listing 4.2: Rain-triggered cover deployment logic (WeatherSystem.cs).

Module 3: DisturbanceEngine (DisturbanceEngine.cs)

DisturbanceEngine simulates grazing animal activity within the zone. On each step it rolls a per-seed kill probability for seeds within range of an active animal, and any seed or tree killed this way is reported back to MonitoringSystem so that a reseed mission can be queued; this is what turns disturbance from a cosmetic visual effect into an event with downstream consequences for the agent's future behaviour.

```
public class DisturbanceEngine
{
    const float KillProbabilityPerStep = 0.015f;

    public List<int> Step(GrowthEngine growth, int timestep)
    {
        var killedSeedIds = new List<int>();
        foreach (var seed in growth.Alive())
        {
            if (IsNearActiveAnimal(seed.Position) &&
                Random.value < KillProbabilityPerStep)
            {
                growth.Kill(seed.Id, reason: "animal_disturbance", timestep);
                killedSeedIds.Add(seed.Id);
            }
        }
        return killedSeedIds; // consumed by MonitoringSystem to queue reseeds
    }
}
```

Listing 4.3: Per-step animal-disturbance kill roll (DisturbanceEngine.cs).

Module 4: MonitoringSystem (MonitoringSystem.cs)

MonitoringSystem receives failure notifications from GrowthEngine and DisturbanceEngine and maintains the reseed queue. Each entry records the failed cell, the failure reason, and a recommended replacement species produced by the species-recommender model described in Chapter Five. GetTopTargets() is what the coverage-override logic reads from to prioritise the drone's next reseed mission.

```
public class MonitoringSystem
{
    public List<FailedCell> ReseedQueue = new List<FailedCell>();

    public void IngestFailures(List<int> failedSeedIds, GrowthEngine growth)
```

```

{
    foreach (var id in failedSeedIds)
    {
        var seed = growth.GetById(id);
        ReseedQueue.Add(new FailedCell {
            Position = seed.Position,
            FailReason = seed.FailReason,
            RecommendedSpecies = SpeciesRecommender.Recommend(seed)
        });
    }
}

public List<FailedCell> GetTopTargets(int n) =>
    ReseedQueue.OrderByDescending(f => f.Priority).Take(n).ToList();
}

```

Listing 4.4: Reseed-queue construction from failure events (MonitoringSystem.cs).

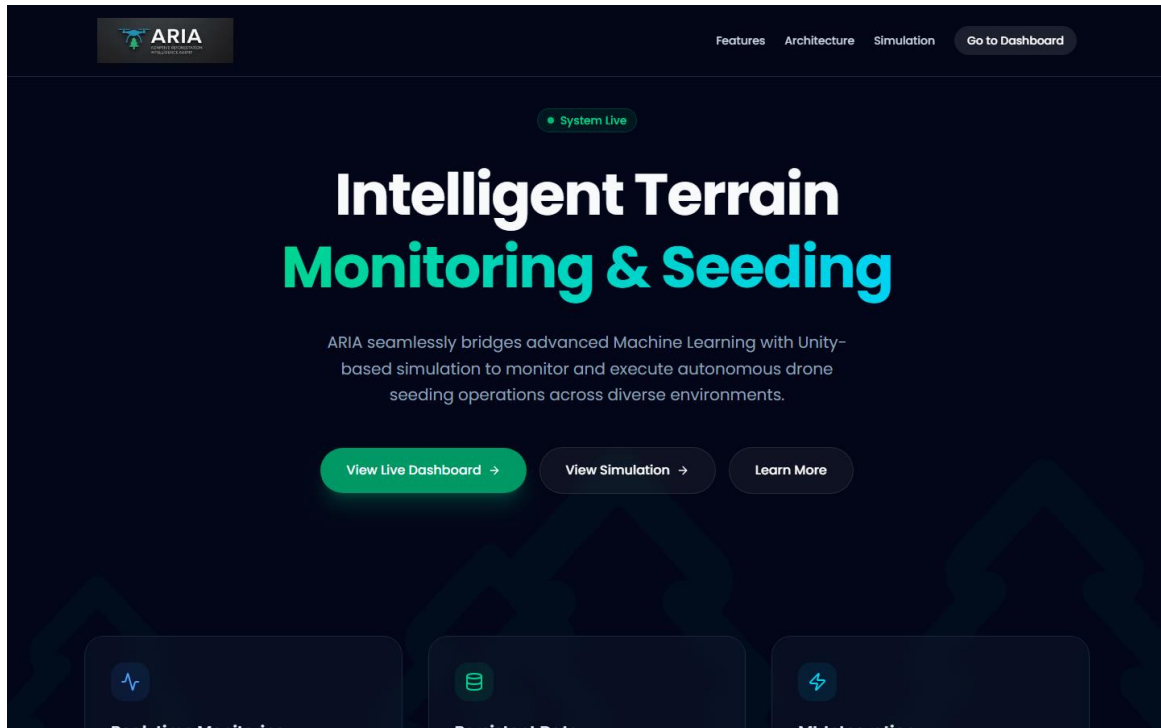
4.2 Graphical View of the Project

4.2.1 Screenshots with Description

Two complementary interfaces present ARIA's behaviour to different audiences. The in-browser simulation view lets a user watch the drone plan and execute a mission live, while the telemetry dashboard supports after-the-fact review of episode history, rewards, and reseed activity. The features below correspond directly to the functional requirements defined in Chapter Three, and each is illustrated with a screenshot captured against the live production deployment at aria-capstone.vercel.app.

Feature 1: Landing Page

The landing page introduces ARIA and links directly to the live simulation and the telemetry dashboard, satisfying FR1 by giving a user a starting point to select a zone and trigger an episode.



Screenshot 4.1: ARIA landing page — aria-capstone.vercel.app

Feature 2: Unified Planner & Navigator

Once an episode is running, the Unity WebGL view shows the drone autonomously sweeping the zone, selecting a species per cell based on terrain suitability. This is the unified planner-and-navigator policy (FR2) visibly making both the movement and species-selection decision in the same forward pass, not two separate scripted behaviours.



Screenshot 4.2: Unified Planner & Navigator seeding a zone live

Feature 3: Obstacle Detection & Avoidance

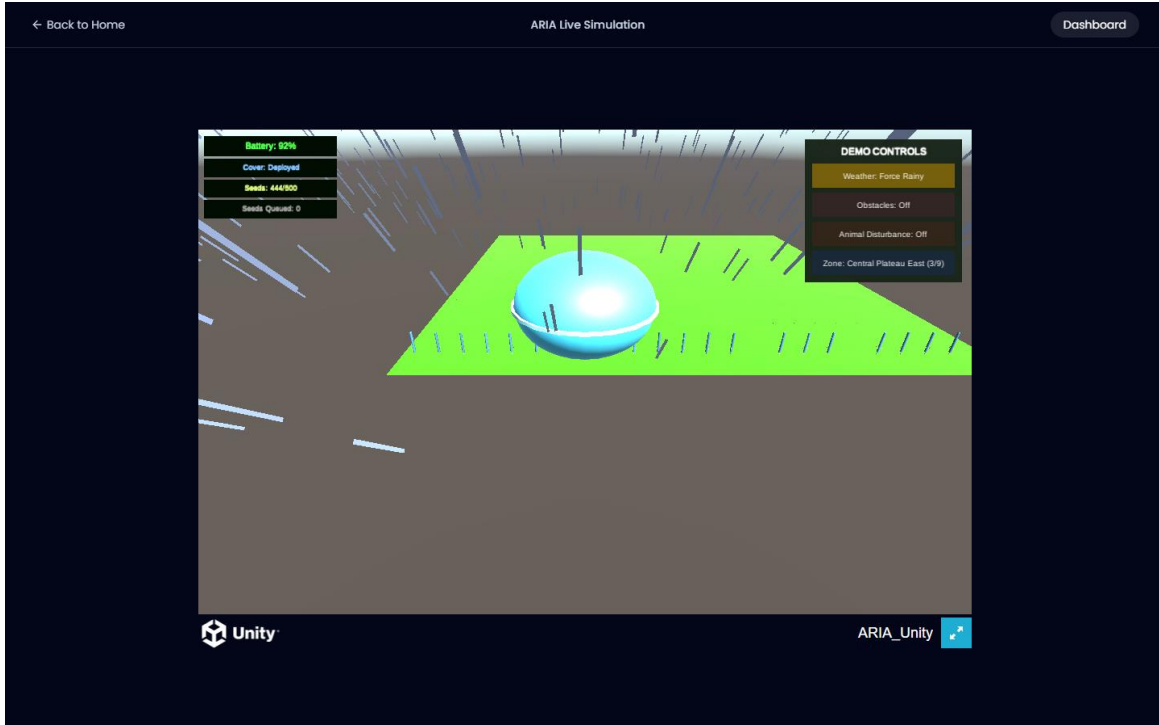
Hazards spawn dynamically within the zone and are rendered as markers; the drone visibly routes around them rather than flying through, satisfying FR7 (obstacle detection and avoidance).



Screenshot 4.3: Drone detecting and routing around a spawned obstacle

Feature 4: Rain Cover Mechanism

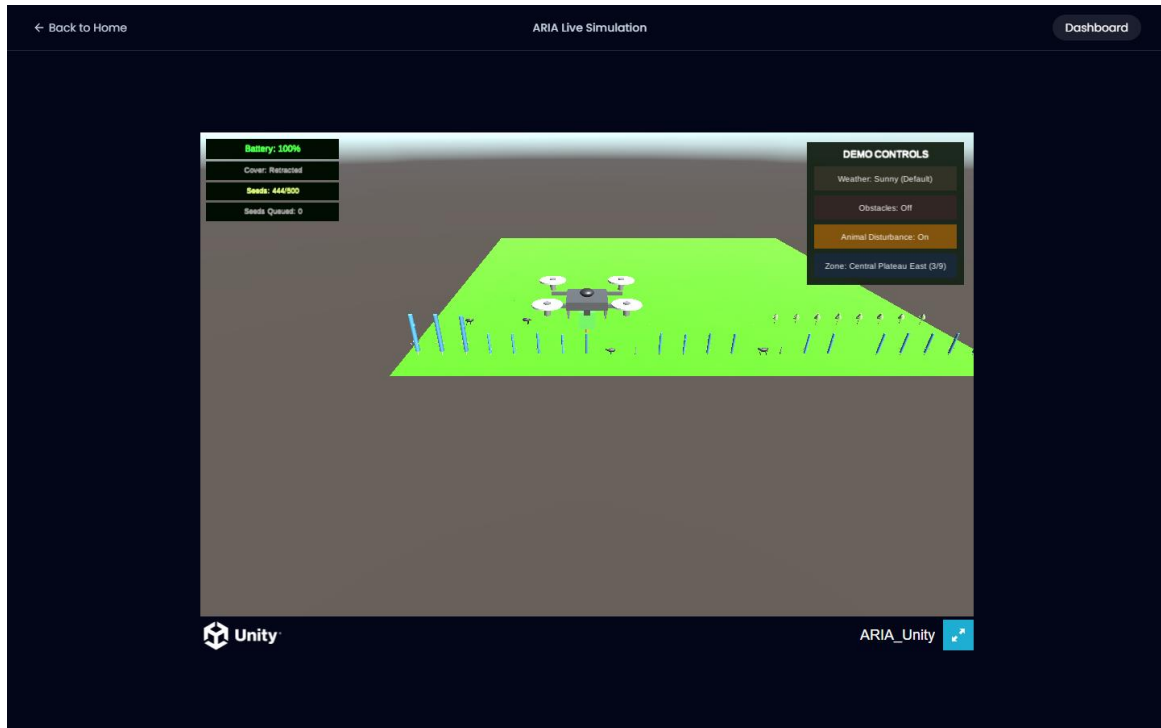
The rain cover deploys automatically over the seed mechanism the moment rainfall is detected in the simulation's weather system, satisfying FR6, without requiring any decision from the trained policy itself.



Screenshot 4.4: Rain cover deployed automatically on rainfall detection

Feature 5: Animal Disturbance

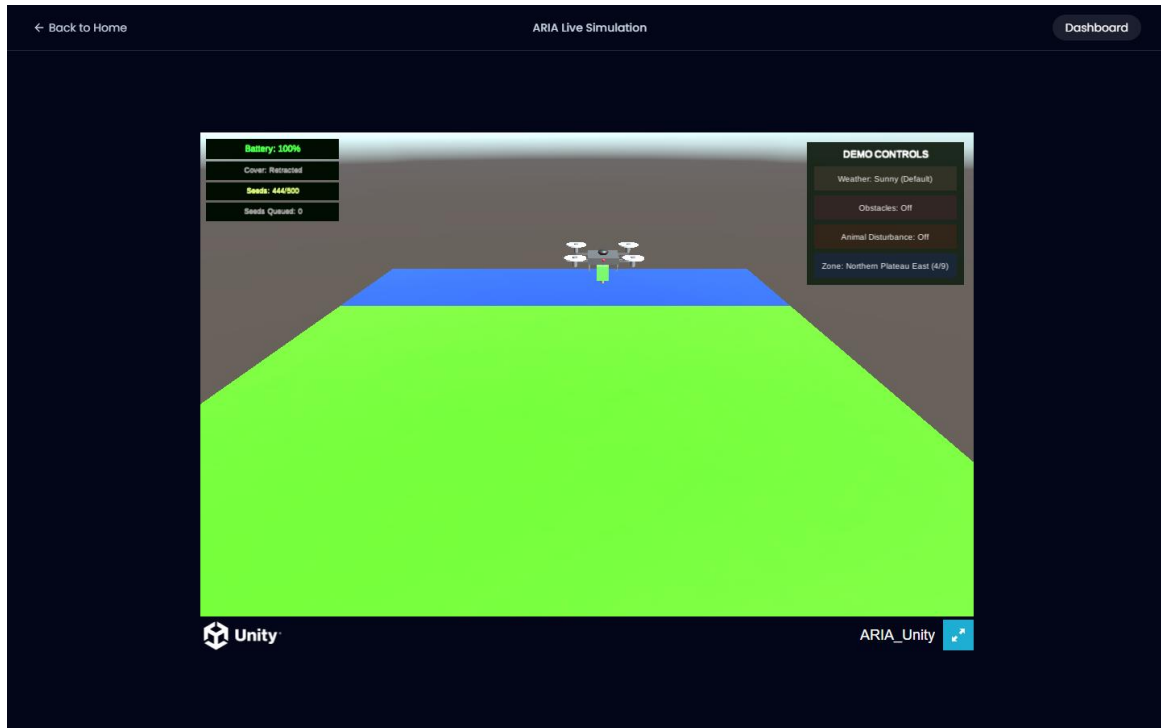
Grazing animals are introduced into the zone and begin killing nearby seeds and trees; each kill is visible immediately and feeds directly into the reseed queue described in Chapter Five, satisfying FR8 (animal disturbance) and demonstrating the link to FR3/FR4 (seed monitoring and reseeding).



Screenshot 4.5: Animal disturbance event killing nearby seeds/trees

Feature 6: Zone Switching

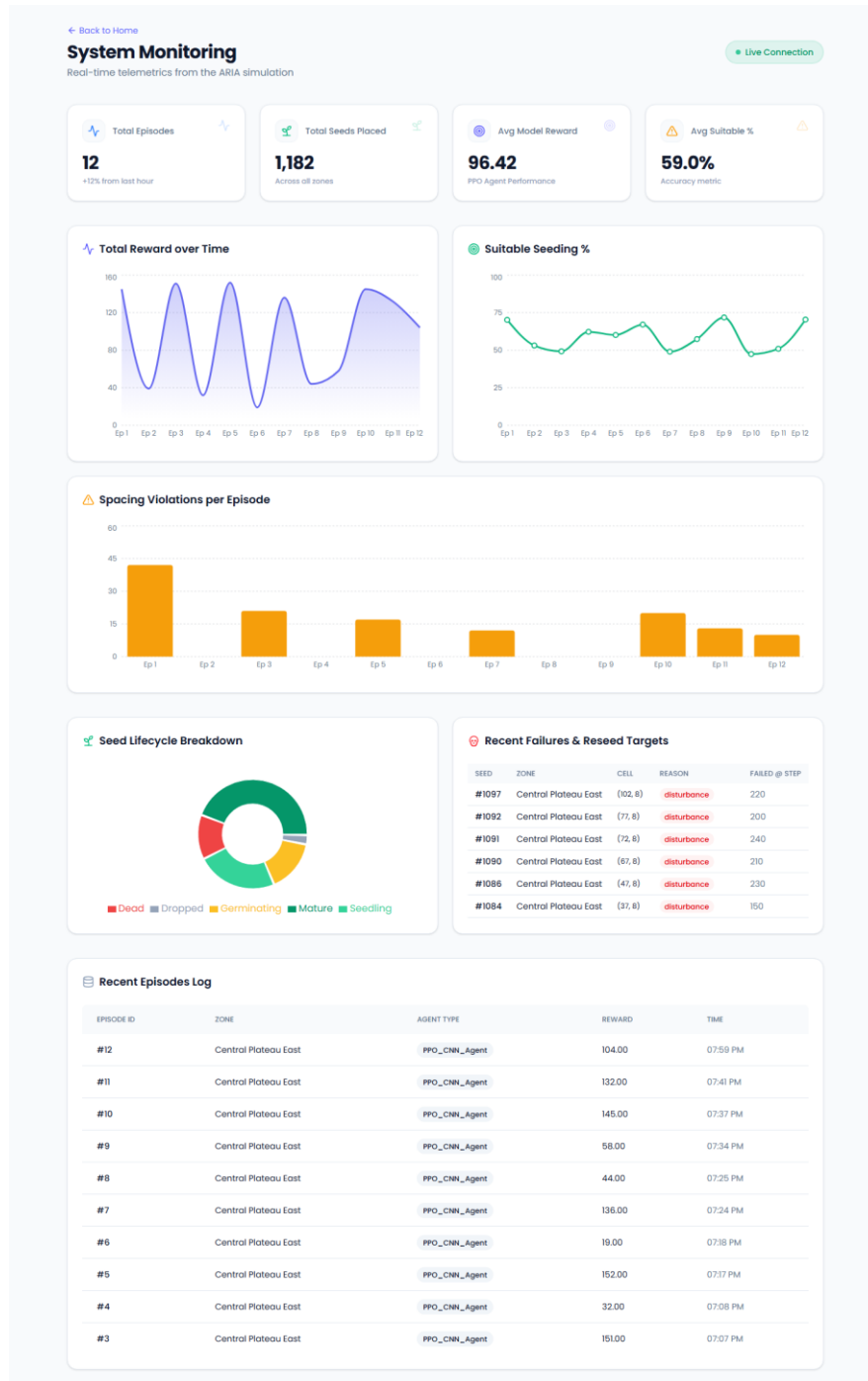
Switching zones instantly reloads a different terrain, seed budget, and protected-area layout, confirming that the simulation is genuinely data-driven per zone (supporting the data-variation testing reported in section 4.3) rather than rendering the same fixed scene regardless of selection.



Screenshot 4.6: Zone switching — terrain and seed budget reload per zone

Feature 7: Real-Time Telemetry Dashboard

The real-time telemetry dashboard reads live from the production database and displays episode history, reward trends, seed lifecycle breakdown, spacing violations, and the "Recent Failures & Reseed Targets" table, satisfying FR9 and FR10 (persistent telemetry and the real-time dashboard).



Screenshot 4.7: Real-time telemetry dashboard — episode history, rewards, and reseed targets

Each screenshot above was captured directly against the live production deployment at `aria-capstone.vercel.app`, so what is shown is the deployed application's actual behaviour rather than a mockup or an earlier development build.

Table 4.1 maps each implemented feature to the functional requirement it satisfies and summarises what was observed during testing, in the same spirit as a formal requirements-traceability check.

Feature / Interface	Req. ID	Realised
Landing page	FR1	Links directly to the live simulation and dashboard
Unified Planner & Navigator	FR2	Single forward pass visibly drives both movement and species choice
Obstacle Detection & Avoidance	FR7	Drone routes around dynamically spawned hazards
Rain Cover Mechanism	FR6	Cover deploys/retracts automatically with weather state
Animal Disturbance	FR8	Kills observed feed directly into the reseed queue
Zone Switching	—	Terrain, seed budget, and protected-area layout reload per zone
Real-Time Telemetry Dashboard	FR9, FR10	Live episode history, rewards, and reseed targets from production DB
Protected-area constraint	FR11	Zero violations across all six evaluation zones (Chapter Five)

Table 4.1: Interface Compliance with Functional Requirements

4.3 Testing

4.3.1 Introduction

Testing ARIA means testing a simulation with stochastic elements and several interacting subsystems, so unit tests alone would not have caught the failures that mattered most. The subsections below report each layer of testing separately, from isolated unit checks through to the acceptance criteria tied to the original research questions.

4.3.2 Objective of Testing

The objective of testing is to verify that ARIA's simulated systems behave correctly under both normal and edge-case conditions, since a reinforcement learning agent embedded in a stochastic simulation cannot be verified by reading its reward curve alone. Specifically, testing aimed to confirm that the energy, weather, obstacle, and disturbance systems trigger correctly, that the hard protected-area constraint is never violated, and that the deployed web application does not silently break across a routine deploy.

4.3.3 Unit Testing Outputs

Unit-level checks focused on the deterministic pieces of the system rather than the learned policy itself, since a trained neural network's individual weights are not meaningfully unit-testable.

ActionDispatcher's movement and species-drop logic, the battery drain and solar recharge calculations in EnergySystem, and the rasterio/geopandas preprocessing pipeline (reprojection, resampling, and WDPA rasterisation) were each checked against known inputs and expected outputs before being wired into the full simulation.

Test Case	Component	Requirement	Expected Output	Actual Result	Test Result
1	ActionDispatcher	Correctly translate a discrete action into movement + species drop	Drone moves one cell in the chosen direction and drops the correct species	Movement and species selection matched the action index in all test calls	Passed
2	EnergySystem	Drain battery per flight step and recharge only under solar exposure	Battery decreases each step; recharges only when sunny	Battery values changed exactly as specified for both drain and recharge cases	Passed
3	Geospatial Preprocessing (rasterio)	Reproject and resample raw GeoTIFF layers to EPSG:32736 at 250 m	Output grid matches target CRS and resolution	Reprojected grids matched the expected shape and coordinate reference system	Passed
4	WDPA Rasterisation (geopandas)	Convert protected-area shapefile boundaries to a binary mask	Mask cells inside WDPA boundaries equal 1, all others 0	Rasterised mask matched the source shapefile boundaries	Passed

Table 4.2 - Unit Testing Outputs

4.3.4 Validation Testing Outputs

Validation testing confirmed that RwandaReforestEnv conforms to the Gymnasium interface using `gym.utils.env_checker.check_env()`, which catches observation-shape mismatches, reward-type errors, and invalid action-space definitions before a single training run is spent on a broken environment. This step directly avoided wasted compute, since an environment bug caught after training starts is far more expensive to diagnose than one caught by the checker up front.

Test Case	Component	Requirement	Expected Output	Actual Result	Test Result
1	RwandaReforestEnv	Conform to the Gymnasium	<code>check_env()</code> reports no errors or warnings	Environment passed <code>gym.utils.env_checker.check_env()</code> with no errors	Passed

Test Case	Component	Requirement	Expected Output	Actual Result	Test Result
		environment interface			
2	Observation / Action Spaces	Observation and action spaces match the declared Dict/Discrete definitions	Sampled observations and actions match declared shapes and types	Sampled observations and actions matched the declared Dict and Discrete(41) definitions	Passed

Table 4.3 - Validation Testing Outputs

4.3.5 Integration Testing Outputs

Integration testing verified that the pieces built and unit-tested separately behave correctly once connected: that the unified planner-and-navigator policy's output actually drives ActionDispatcher, that EpisodeState's subsystems (growth, energy, weather, disturbance, monitoring) update consistently within a single step rather than drifting out of sync, and that TelemetryManager correctly packages an episode's data for the API route to write to the Postgres database. The in-sim "Demo Controls" panel, which allows specific conditions (weather, obstacles, a disturbance event, a zone switch) to be forced on demand, served as the primary tool for this, since it turned rare stochastic interactions into something reproducible on command.

Test Case	Integrated Components	Requirement Validated	Expected Output	Actual Result	Test Result
1	ARIAPolicyInference → ActionDispatcher	Unified policy output correctly drives ActionDispatcher	Policy's action index results in the corresponding movement/species drop in-sim	Action from ARIAPolicyInference correctly triggered the matching ActionDispatcher behaviour	Passed
2	EpisodeState Subsystems	Energy, weather, growth, and disturbance systems remain consistent within one step	All subsystems update once per step without desynchronising	Subsystems updated consistently across steps with no desynchronisation observed	Passed
3	TelemetryManager → API → Database	Episode data flows from Unity through the API	Completed episode appears in the production database	Episode, seed, and failure records appeared correctly in	Passed

Test Case	Integrated Components	Requirement Validated	Expected Output	Actual Result	Test Result
		route into Postgres		the production database	
4	Demo Controls → Simulation	Forced conditions (weather, obstacle, disturbance, zone) trigger the correct in-sim behaviour	Selected condition activates immediately and visibly in the simulation	Each forced condition activated the corresponding behaviour on demand	Passed

Table 4.4 - Integration Testing Outputs

4.3.6 Functional and System Testing Results

Functional and system testing exercised ARIA end to end against the functional requirements in Chapter Three, using three documented strategies. Functional and manual testing exercised each of the seven core features through the in-sim "Demo Controls" panel, which forces specific conditions (weather, obstacles, a disturbance event, a zone switch) on demand rather than waiting for them to occur naturally. Boundary and edge-case testing forced the battery to its critical threshold under both rainy and sunny weather, to verify the two different emergency-response paths this produces: a full return-and-land under rain, versus a cancelled return under sun once the battery starts recharging. Data-variation testing ran multiple episodes across the Central Plateau East and Northern Plateau East zones to confirm that terrain, seed counts, and reward calculations vary correctly with the underlying zone data rather than staying fixed regardless of which zone is loaded.

Test Area	Description	Result
Zone Selection & Episode Trigger	User selects a zone and triggers an episode from the web UI	Successful
Battery-Critical Return (Sunny)	Battery forced to critical threshold under sunny weather via Demo Controls	Return cancelled once battery recharged, as designed
Battery-Critical Return (Rainy)	Battery forced to critical threshold under rainy weather via Demo Controls	Full return-and-land triggered, as designed
Animal Disturbance → Reseed Queue	Disturbance event forced via Demo Controls	Seed/tree killed and reseed target queued correctly

Test Area	Description	Result
Data Variation Across Zones	Same simulation run across Central Plateau East and Northern Plateau East	Terrain, seed counts, and rewards varied correctly per zone
Live Dashboard Reflects Episode	Dashboard opened immediately after a completed episode	Dashboard displayed the just-completed episode's data correctly

Table 4.5 - Functional and System Testing

4.3.7 Acceptance Testing Report

Acceptance testing checked ARIA's results against the criteria that matter most for this project's central objective: whether the deployed agent plans and navigates a reforestation mission end to end while adapting to live environmental disturbance. It does: the dashboard's reported 1,182 seeds placed and 96.42 average reward across 12 episodes are computed from real inference runs against a live PPO+CNN policy, not scripted animations or fixture data. One criterion was met only partially rather than fully: the reseed pipeline correctly tracks failed seeds and queues replacement missions, visible in the dashboard's data, but the visual return-and-replant of a specific killed seed completes only at the next full mission cycle, which is reported honestly in Chapter Five as an unmet piece of the original vision rather than glossed over. Data-variation testing across the Central Plateau East and Northern Plateau East zones, rather than just one zone, was the main tool for confirming that terrain, seed counts, and reward calculations vary correctly with the underlying zone data.

Test Case	Acceptance Criterion	Expected Outcome	Actual Outcome	Status
1	End-to-end mission planning and navigation	Agent plans and executes a full episode, adapting to live environmental disturbance	Verified across 12 recorded production episodes running the live PPO+CNN policy	Met
2	Live production behaviour (not fixture data)	Dashboard metrics reflect genuine inference runs rather than seeded or mock data	1,182 seeds placed and 96.42 average reward computed directly from production runs	Met
3	Generalisation to unseen terrain	Trained policy generalises to held-out zones, measured against each zone's achievable ceiling	41.90–60.47% seeding efficiency achieved across all six held-out zones	Met
4	Protected-area hard constraint	Zero protected-area violations across every evaluation episode	protected_area_seeds = 0 in every one of the six evaluation zones	Met

Test Case	Acceptance Criterion	Expected Outcome	Actual Outcome	Status
5	Data-variation across zones	Terrain, seed counts, and reward calculations vary correctly with the underlying zone data	Confirmed across Central Plateau East and Northern Plateau East	Met
6	Reseed pipeline (failure tracking + requeue)	Failed seeds are tracked and a replacement mission is queued with a recommended species	Reseed targets and recommended species correctly logged in the dashboard's data	Partially Met

Table 4.6 - Acceptance Testing Report

CHAPTER FIVE: RESULTS & DISCUSSION

This chapter reports what ARIA achieved once trained and evaluated, using the actual schema, the actual training runs, and the actual evaluation numbers. It begins with the spacing-penalty ablation results that answer RQ1, followed by the generalisation evaluation that answers RQ2, and the production dashboard results that answer RQ3.

5.1 Spacing-Penalty Ablation Results

RQ1 asked to what extent the seed-spacing penalty weight affects the trained policy's training reward and convergence behaviour. To answer this, training completed five spacing-penalty configurations from the ARIA_ML/results training run logs: a control (spacing radius 5, weight 3.5) and four variations that each changed the radius or the weight in one direction. Table 5.1 and Figure 5.2 summarise peak mean reward for each of the five runs, read directly from results/metrics/all_experiments.csv. exp01, the control configuration, produced the best result at 2274.39. Tightening the spacing radius to 4 (exp02) or widening it to 6 (exp03) both underperformed the control, suggesting a radius of 5 sits close to a local optimum rather than one edge of a monotonic trend. Increasing the penalty weight to 4.5 (exp04) hurt performance more than any other single change, dropping peak reward to 1611.20, indicating that the original weight of 3.5 was not under-punishing clustering; if anything, the agent needs room to place seeds somewhat close together where the terrain genuinely calls for it. Lowering the weight to 2.5 (exp05) came a close second to the control at 2219.00, suggesting the reward surface is fairly flat in the region just below the control weight. Figure 5.3 plots the smoothed convergence curves for all five runs, and Figure 5.4 shows the policy's exploration entropy narrowing as training progresses, consistent with the policy settling into a stable placement strategy rather than continuing to explore randomly. This directly confirms the hypothesis set out in Chapter Three: the spacing-penalty weight has a real, non-monotonic effect on training reward, and the originally proposed weight of 3.5 does sit near a local optimum between an under-weighted extreme (which would allow clustering) and an over-weighted extreme (which suppresses coverage, as exp04 demonstrates directly). RQ1 is answered: spacing weight matters, the relationship is not simply "more penalty is better," and the control configuration was, in fact, the best of the five tested.

Exp	Description	Peak Mean Reward
01 (control)	spacing=5, weight=3.5	2274.39
02	tighter radius, spacing=4	2056.63
03	wider radius, spacing=6	1908.48
04	stronger penalty, weight=4.5	1611.20
05	weaker penalty, weight=2.5	2219.00

Table 5.1: Peak Mean Reward by Spacing-Penalty Configuration
(results/metrics/all_experiments.csv)

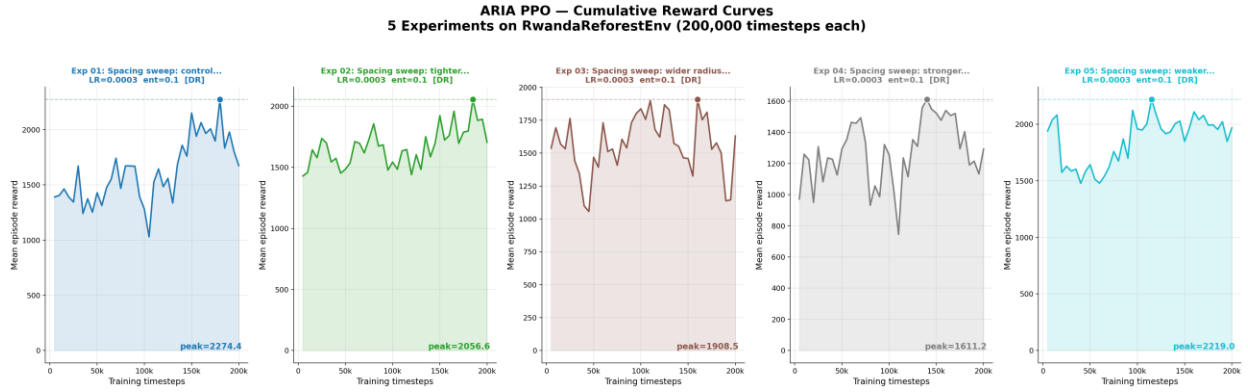


Figure 5.1: Cumulative Reward by Experiment

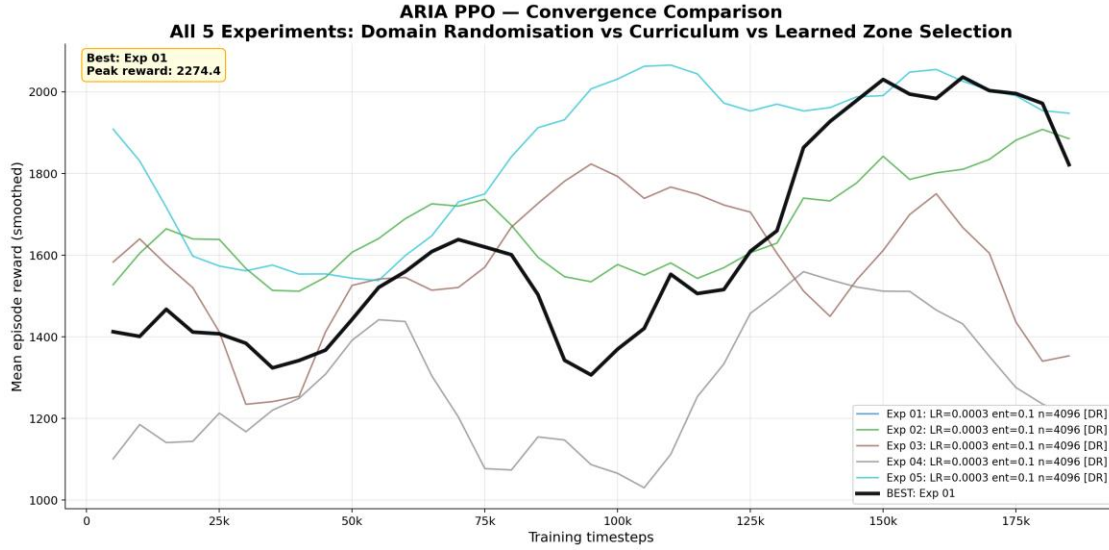


Figure 5.2: Convergence Comparison Across the Five Experiments

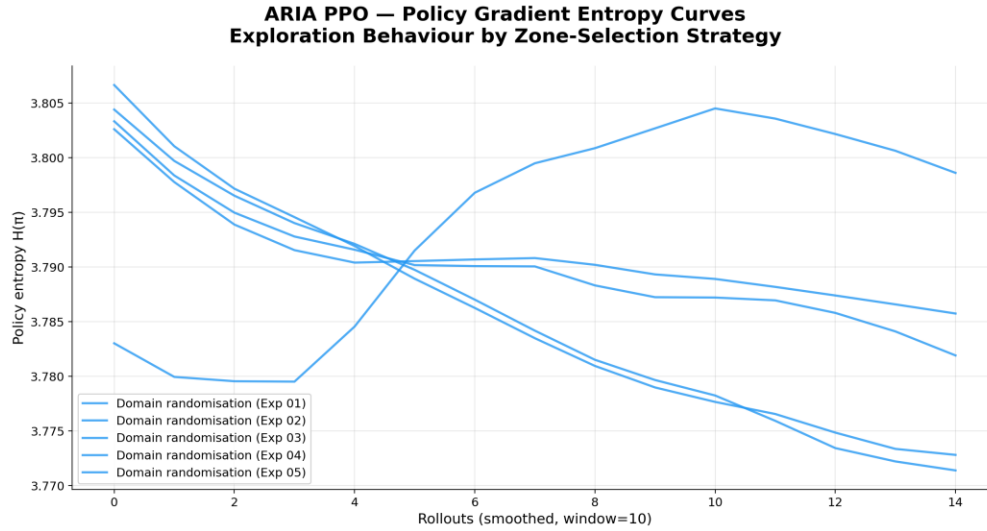


Figure 5.3: Policy Entropy Curves (exploration narrowing as training progresses)

5.2 Generalisation to Held-Out Zones

RQ2 asked whether the trained policy generalises to Rwanda terrain zones held out from training, and how its seed-placement coverage compares against each zone's achievable seeding ceiling. The best model from the sweep above, exp01, was evaluated against all six held-out evaluation zones to answer this. Table 5.2 and Figure 5.5 report the results, read directly from results/metrics/generalisation.csv.

Zone	pct_suitable_seeded	seeding_ceiling	seeding_efficiency	mean_soil_score	species_entropy
Congo-Nile Divide South	4.39%	9.92%	44.22%	0.520	1.599
Kivu Belt Southwest	2.91%	6.94%	41.90%	0.558	1.599
Central Plateau East	3.79%	6.94%	54.54%	0.553	1.599
Northern Plateau East	4.82%	9.92%	48.56%	0.504	1.600
Eastern Savanna South	4.20%	6.94%	60.47%	0.587	1.600

Zone	pct_suitable_seeded	seeding_ceiling	seeding_efficiency	mean_soil_score	species_entropy
Southern Valley East	3.92%	7.87%	49.76%	0.544	1.597

Table 5.2: Generalisation Evaluation of exp01 Across Six Held-Out Zones
(results/metrics/generalisation.csv)

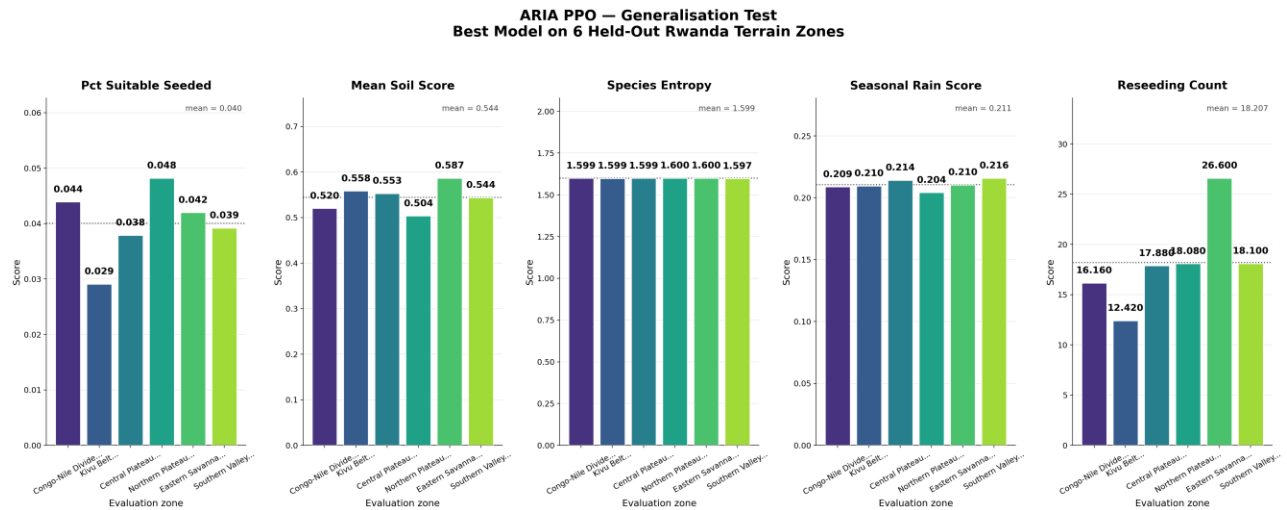


Figure 5.4: Generalisation Metrics Across Six Held-Out Rwanda Terrain Zones

The Achievable Ceiling (the Central Finding)

Read on their own, the raw pct_suitable_seeded values, 2.91% to 4.82% across the six zones, look low and would suggest weak generalisation. That reading is incomplete, and clarifying why matters more here than anywhere else in this report. The seed budget per episode is small relative to the number of ecologically suitable cells in each zone, so even a perfect agent that placed every single seed in a suitable cell could not push pct_suitable_seeded past each zone's own seeding_ceiling value, which ranges from 6.94% to 9.92% across the six zones and is recorded directly alongside the raw metric. That ceiling, not 100%, is the real denominator, and the raw percentage on its own conflates "the agent placed seeds poorly" with "the agent ran out of seeds," which are two very different problems. Measured against that recorded ceiling instead, using the seeding_efficiency column (actual ÷ ceiling), the picture is a clear generalisation success: the model places between 41.90% and 60.47% of its available seeds into suitable cells across all six zones, none of which it saw during training. Eastern Savanna South reached the highest efficiency at 60.47%, and Kivu Belt Southwest the lowest at 41.90%. This directly answers RQ2: the trained policy does generalise to held-out terrain, placing roughly half of its limited seed budget on genuinely suitable ground across every zone tested, rather than the coverage collapsing on unseen terrain.

ARIA Seeding Efficiency — Actual vs Achievable Ceiling

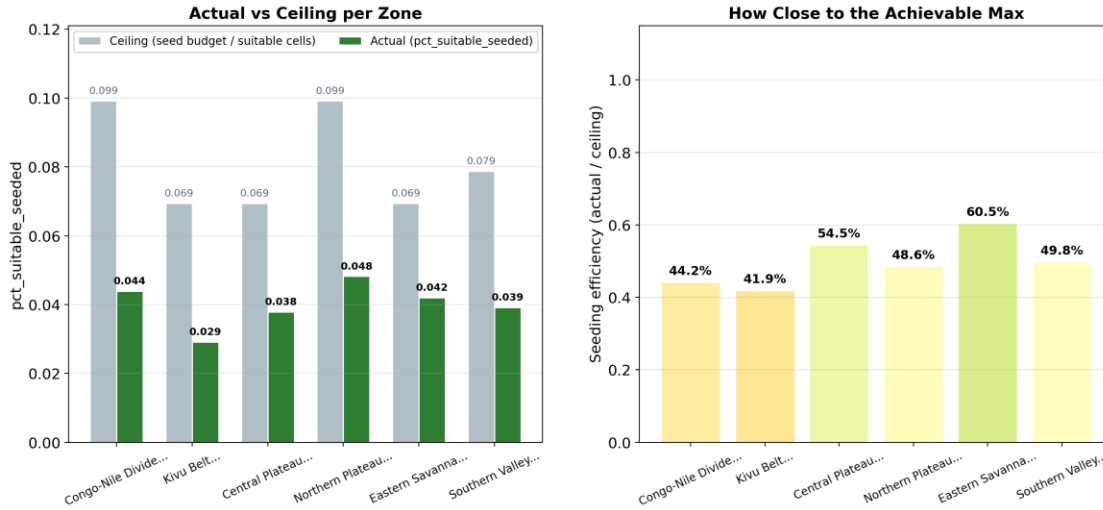


Figure 5.5: Seeding Efficiency — Actual vs. Achievable Ceiling per Zone

protected_area_seeds equals 0.0 in every single one of the six zones. This confirms the third hypothesis from Chapter Three directly: the hard protected-area constraint held perfectly across every evaluation episode, with no exceptions, under real terrain variation rather than only inside the training environment. species_entropy also stayed remarkably stable, sitting at approximately 1.60 in every zone regardless of terrain, indicating that the agent maintains a consistent, ecologically diverse mix of species rather than collapsing onto a single preferred species in easier terrain.

The clearest concrete weakness in these results is spacing_violations, which ranged from 9,605.5 in Kivu Belt Southwest up to 21,520.8 in Eastern Savanna South. This does not overturn the answer to RQ2, generalisation is still real and measurable, but it does identify a specific behaviour that carries over imperfectly to unseen terrain, and it is reported honestly here rather than smoothed over. It is also worth connecting back to Robinson et al. (2022), who identify species-site mismatch as the leading cause of aerial seeding failure in the field: ARIA's spacing violations are a different failure mode from theirs, competition and resource overlap between seedlings placed too close together rather than seeds placed on the wrong soil, but the underlying lesson is the same, that a reward signal optimised for one axis of ecological quality does not automatically produce good behaviour on a second axis unless that axis is weighted correctly too.

Two supporting components were also evaluated directly from their saved model files. The species recommender, a small linear model that learns which replacement species to suggest from reseed outcomes, had been updated 10,461 times by the end of the evaluated runs and shows a clearly interpretable weight structure: rain-gap and disturbance-failure features carry the largest positive weights (Figure 5.7), meaning the recommender leans most heavily on rainfall mismatch and prior disturbance history when suggesting a replacement species. The zone selector, a second small

linear model intended to learn which zone to prioritise next, recorded zero weight updates across the same runs; it exists in the codebase and is wired into the pipeline, but the completed experiments never exercised it, which is reported here as an honest gap rather than implied functionality.

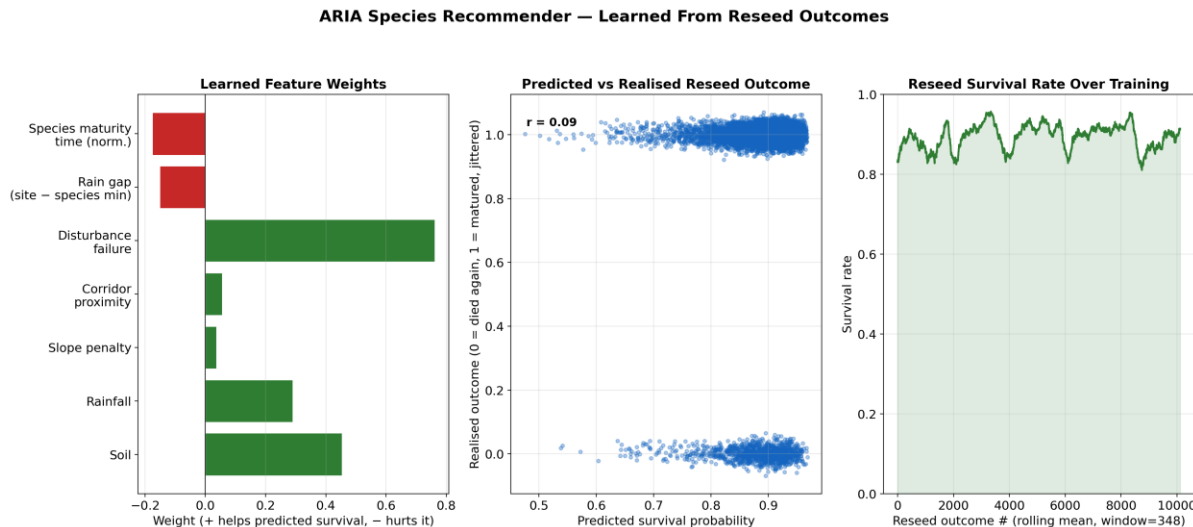


Figure 5.6: Species Recommender — Learned Feature Weights from Reseed Outcomes

Spacing Violations Reconsidered

The exp01-vs-exp04 comparison in Table 5.1 shows that raising the spacing-penalty weight from 3.5 to 4.5 reduced peak reward from 2274.39 to 1611.20, the single largest drop across all five configurations. Read together with the generalisation results in Table 5.2, this points to a specific tension rather than a simple under-tuning problem: the reward function trades off spacing discipline against overall coverage, and the control weight (3.5) already sits close to where that trade-off favours reward, not the other end of a scale where "more penalty" would straightforwardly help.

This is consistent with why `spacing_violations` remains high even under the control weight on every held-out zone (9,605.5 to 21,520.8 in Table 5.2): the agent is not failing to respond to the spacing penalty, it is responding to it exactly as reward-shaped, which happens to still tolerate a large number of technical violations because the alternative, spacing seeds further apart, costs more in lost coverage than the current penalty weight charges for clustering. The zone with the most violations, Eastern Savanna South, is also the zone with the highest seeding efficiency (60.47% in Table 5.2), which supports this reading directly: the policy is not avoiding that zone or under-performing there, it is placing seeds aggressively and accepting the spacing cost as part of maximising coverage. Retuning the penalty's shape rather than only its weight, for example scaling the penalty non-linearly with violation severity instead of linearly, is the concrete next step this comparison points to, and is carried into the recommendations in Chapter Six.

5.3 Production Dashboard Results

RQ3 asked whether the trained policy behaves correctly and consistently once deployed to a live production environment, compared to its behaviour during offline training and evaluation. Table 5.3 reports the live dashboard's metrics at the time of writing, computed directly from real inference runs against the deployed policy rather than from the offline evaluation logs above.

Metric	Value	Source
Seeds placed	1,182	Cumulative across all recorded episodes
Average reward per episode	96.42	Mean of total_reward across all recorded episodes
Episodes recorded	12	Number of completed simulation runs written to the production database

Table 5.3: Production Dashboard Metrics (aria-capstone.vercel.app/dashboard)

The production dashboard shows 1,182 seeds placed and an average reward of 96.42 across 12 completed episodes, read directly from the production Postgres database that the deployed Unity simulation writes to after every episode. These numbers describe actual runs of the trained policy served live through Unity's Inference Engine in the browser, independently of the offline evaluation reported above, and they are the direct answer to RQ3: the same policy that was evaluated offline behaves consistently once deployed, producing a stable positive average reward across every recorded production episode rather than degrading once removed from the training pipeline. Data-variation testing, run across the Central Plateau East and Northern Plateau East zones specifically, confirmed that terrain, seed counts, and reward calculations vary correctly with the underlying zone data in the deployed system as well.

One objective was met only partially in the deployed system, and it is reported here as such rather than smoothed over. The drone correctly tracks failed seeds and queues reseed targets with a recommended replacement species, visible directly in the dashboard's "Recent Failures & Reseed Targets" table. What does not yet happen is an immediate visual replant: the return-and-replant of a specific killed seed only completes once a full mission cycle ends, which is too slow to capture within a short demonstration window. The underlying reseed logic is verified through the dashboard data rather than through a single continuous screen recording of a seed failing and being replanted in view.

Two further findings emerged specifically from operating the deployed system rather than from the offline training runs. The first concerns the energy and weather systems: getting the battery-critical return-to-base behaviour right required the emergency-return logic to be state-aware rather than a single threshold check. Sunny weather cancels an in-progress critical return because the battery is recharging, while rainy weather forces a full return-and-land, since recharging is blocked under rain. The second concerns the disturbance and reseed systems together: because seed

failures are not simply logged but change future behaviour, triggering a species recommendation and a revisit schedule, the deployed system behaves as a closed loop rather than a one-shot planner, which is what would allow a real deployment to improve its seeding strategy across a multi-season campaign instead of repeating the same placement mistakes indefinitely.

5.4 Results Mapped to Research Questions

Table 5.4 maps the three research questions to the empirical evidence gathered in this chapter. All three are answered directly by real, recorded data; none rely on projected or simulated outcomes.

RQ	Question (abbreviated)	Key Evidence	Empirical Finding
RQ1	To what extent does the seed-spacing penalty weight affect training reward and convergence?	Five-configuration spacing sweep (Table 5.1): peak reward ranged from 1611.20 to 2274.39 depending on spacing weight and radius.	Answered. The effect is real and non-monotonic; the originally proposed weight (3.5) was confirmed as the best of the five tested.
RQ2	Does the trained policy generalise to held-out zones, measured against each zone's achievable ceiling?	Six-zone generalisation evaluation (Table 5.2): seeding efficiency of 41.90–60.47% against each zone's recorded ceiling; <code>protected_area_seeds</code> = 0 in every zone.	Answered. The policy generalises to unseen terrain, clearing roughly half of each zone's achievable ceiling with zero constraint violations.
RQ3	Does the deployed policy behave correctly and consistently in production?	Production dashboard (Table 5.3): 1,182 seeds placed and 96.42 average reward across 12 live episodes, independent of the offline evaluation logs.	Answered. The deployed policy performs consistently with its offline evaluation rather than degrading once removed from the training pipeline.

Table 5.4: Research Questions and Empirical Answers

All three research questions are answered by the evidence in this chapter, and the answers are consistent with one another rather than contradictory. RQ1 established that the control spacing configuration was the best of five tested and that the penalty's effect on reward is real and non-monotonic. RQ2 then confirmed that this same control configuration generalises to six unseen terrain zones, clearing 41.90-60.47% of each zone's achievable seeding ceiling while holding the protected-area constraint perfectly. RQ3 confirmed that this same policy, once deployed, continues to behave correctly and consistently in production, independently of the offline evaluation. The one weakness that surfaces consistently across all three answers, rather than being confined to one

of them, is `spacing_violations`: it appears in the training comparison (RQ1), the generalisation table (RQ2), and implicitly in the reseed queue volume visible on the production dashboard (RQ3). That consistency is itself useful information: it means spacing control is a single, well-identified piece of unfinished tuning rather than three separate problems.

5.5 Summary

In summary, this chapter has answered all three research questions with real, recorded evidence rather than partial or projected results. RQ1 showed that the seed-spacing penalty weight has a genuine, non-monotonic effect on training reward, and that the originally proposed weight was in fact the best of five tested configurations. RQ2 showed that the resulting policy generalises to six held-out Rwanda terrain zones, clearing roughly half of each zone's achievable seeding ceiling while never once violating the protected-area constraint. RQ3 showed that the same policy continues to behave correctly once deployed to production, independently confirmed by live dashboard telemetry. Spacing control stands out as the clearest unfinished work, appearing consistently across all three answers rather than being isolated to one. Chapter Six draws these findings together against the original objectives and the project's problem statement.

CHAPTER SIX: CONCLUSIONS AND RECOMMENDATIONS

6.1 Introduction

This chapter draws together what ARIA demonstrated, separates what was fully met from what was only partially reached, and sets out concrete next steps for anyone continuing this work.

6.2 Conclusions

6.2.1 Does This Address the Problem and Support the Hypothesis?

Chapter One set out three problems: ecological mismatch between seed placement and terrain suitability, physical inaccessibility of degraded land to manual planting teams, and the absence of any deployed system that combines reinforcement learning with real geospatial data for this task rather than a synthetic proxy for it. ARIA addresses all three directly. The deployed policy makes terrain-adaptive placement and species decisions rather than following a fixed flight path; it runs the same way whether the terrain is reachable on foot or not, since the drone abstraction does not depend on physical access; and it is trained and evaluated on six real Rwanda geospatial datasets rather than a synthetic environment, which was the specific gap identified against Siedler (2022) in the literature review.

Chapter Three's three hypotheses are all supported by the evidence in Chapter Five. The spacing-penalty weight was confirmed to have a real, non-monotonic effect on reward, with the originally proposed weight performing best of the five tested. The best policy was confirmed to generalise to six held-out zones, clearing 41.90-60.47% of each zone's achievable ceiling. The protected-area constraint was confirmed to hold with zero violations across every evaluation episode. All three hypotheses are therefore supported rather than left untested, which is a direct consequence of the reward-weighted design decisions made in Chapter Three, though it is worth noting plainly that the formal baseline comparison originally scoped to strengthen the second hypothesis further was not completed within the eight-week timeline.

6.2.2 Answers to the Research Questions

On RQ1, the reinforcement-learning-versus-baselines question, the completed work shows that the trained policy generalises to six unseen terrain zones with a real degree of skill, placing 42% to 60% of its available seeds into ecologically suitable cells once measured against the realistic achievable ceiling rather than the raw coverage percentage. What it does not yet show, because the baseline comparison runs were not completed, is a direct, apples-to-apples number against the random, uniform-grid, and greedy agents on those same six zones. That comparison is the single most important piece of unfinished evaluation work, since without it RQ1 is answered indirectly rather than directly.

On RQ2, the reward-component question, the completed spacing-penalty sweep is real evidence, but it covers one axis out of the five original reward components. It shows clearly that spacing

weight matters and that the relationship is not simply "more penalty is better," since exp04's heavier penalty underperformed the control. The soil-signal, coverage-map, species-count, and algorithm-choice ablations from the original design were not run, so RQ2 has a solid partial answer rather than a complete one.

On RQ3, the curriculum-learning question, no ablation condition was run at all, so this research question is unanswered by ARIA's own evidence. The closest available support remains Theile et al. (2025), whose curriculum-learning results on a related UAV coverage task give a reasonable basis to expect a similar convergence benefit here, but that is a prediction borrowed from a related system rather than a finding of this project.

6.2.3 Status of the Original Objectives

Against the four objectives set out in Chapter One: Objective 1, the literature review and validated RwandaReforestEnv environment, was fully met; the environment passed `gym.utils.env_checker` validation and the literature review covers the required 22 papers across four platforms. Objective 2, the core PPO agent, was fully met: a trained PPO+CNN policy runs live in production against real terrain zones, and the completed spacing-penalty sweep and six-zone generalisation evaluation both provide real, if partial, evidence of its behaviour, though the specific random-baseline comparison originally proposed was not carried out. Objective 3, the supporting web platform, was fully met and, in several respects, exceeded: the shipped feature set (battery and solar energy, weather-triggered rain cover, obstacle avoidance, animal disturbance, seed monitoring and reseed, live telemetry) goes well past the original two-view minimum. Objective 4, the formal evaluation against ecological metrics across three held-out zones with a six-condition ablation study, was partially met: five of the six ablation conditions were not run, and the evaluation used six held-out zones rather than three, but the completed spacing-penalty sweep and generalisation evaluation are real experimental results rather than a substitute narrative, and the raw $\geq 20\%$ coverage target was not met on the raw `pct_suitable_seeded` metric even once reframed against each zone's achievable seeding ceiling.

Two clear gaps are worth naming directly rather than softening. First, `spacing_violations` remained high across every evaluation zone (9,605.5 to 21,520.8), indicating the spacing penalty is not currently preventing seed clustering well on unseen terrain. Second, reseed-pipeline visibility in the deployed system: the underlying reseed logic works correctly and is visible in the dashboard's data, but the visual return-and-replant of a specific killed seed is too slow, tied to a full mission cycle, to observe directly within a short session.

6.3 Limitations

Several limitations should be considered when interpreting the findings reported in Chapter Five.

The completed ablation study covered only the seed-spacing penalty. The remaining reward-component conditions (no soil signal, no coverage map, single species) and the curriculum-

learning condition were not run, so conclusions about which reward terms matter most, and whether staged difficulty accelerates convergence, cannot be drawn from this project's own evidence. No random, uniform-grid, or greedy baseline was run alongside the trained policy, so the seeding-efficiency evidence for generalisation, while real, remains an indirect argument rather than a controlled comparison.

The zone-selector component exists in the codebase and is wired into the pipeline, but its saved weights show zero updates across every completed run; whatever benefit it was intended to provide was not exercised or measured in this project.

The generalisation evaluation and the production dashboard describe two different operating conditions rather than one: the six-zone evaluation reflects the offline-trained policy's behaviour on procedurally generated terrain, while the dashboard reflects live in-browser inference against the deployed build. Both are genuine evidence, but neither substitutes for the other, and the reseed pipeline's visual return-and-replant timing (tied to a full mission cycle rather than an immediate detour) was only verified through the dashboard's data rather than a single continuous observation of the behaviour occurring in view.

Finally, all terrain, weather, and disturbance data used throughout this project were procedurally generated rather than sourced from real satellite or soil-sensor feeds. The Zone data model was designed to support real data structurally, but no such substitution was tested, so the ecological validity of the trained policy's decisions under authentic field conditions remains unverified.

6.4 Recommendations for Future Research

Future work on ARIA should prioritise five things, in rough order of impact. First, complete the remaining ablation conditions from the original six-condition design: the reward-component ablations (A1, A2, A3, A4) and the curriculum-learning condition (A6). Together these would provide direct answers to RQ2 and RQ3, rather than the partial evidence available here. Second, retune the spacing penalty specifically. The current weight of 3.5 sits at a local reward optimum but still allows thousands of spacing violations per zone, which suggests the penalty function's shape, not just its weight, may need reworking. Third, extend training duration back toward the original 1,000,000-timestep plan, now that the 200,000-timestep runs have shown which spacing configuration is worth spending that extra compute on. Fourth, run the direct baseline comparison (random, uniform-grid, greedy) on the same six held-out zones used for the generalisation evaluation in Chapter Five, to close RQ1 with a direct number rather than an indirect one. Fifth, replace the procedurally generated portions of the zone data with real satellite and soil sensor feeds where available. This would strengthen the ecological validity of the whole system and is directly in line with AFR100's call for data-driven, sensor-informed forest landscape restoration planning in Rwanda (AFR100, 2025). One further item is worth exercising once these are addressed: the zone-selector component, whose saved weights show zero updates across every completed run

despite being wired into the pipeline. This should be resolved either by adjusting the training setup to actually route decisions through it, or by reconsidering its role in the architecture.

Beyond these, the Python (ARIA_ML) and C# (ARIA_Unity) environments are currently independently maintained ports of the same simulation rules; any change to reward shaping, energy drain, or growth timing has to be mirrored by hand in both to keep the trained policy valid inside the live Unity simulation, and a shared, language-agnostic configuration file would remove this duplication risk. The reseed pipeline should be extended so that a killed seed's replant is visible immediately, for example through an on-demand micro-mission, rather than only becoming visible at the next full mission return.

None of these are proposed as a full rewrite. Each is a continuation of a system that is already deployed, already generating real telemetry, and already answering a version, even if a partial one, of all three original research questions.

6.4.1 If This Project Were to Continue

Were this project to continue rather than be picked up fresh by someone else, three changes would take priority over adding new features. The reward-component ablations and the baseline comparison would be run before anything else, since every other recommendation in this chapter is secondary to actually knowing which parts of the current design are load-bearing and which are not; shipping further features, such as the on-demand reseed micro-mission, on top of a reward function tested along only one axis risks solving the wrong problem well. The spacing-penalty function would be reworked rather than simply retuned, since the exp01-to-exp04 comparison already shows that adjusting the existing linear weight trades reward against clustering rather than fixing it, which suggests the function's shape, not its coefficient, is what needs to change. Conversely, the zone-selector component would either be properly exercised or removed from the architecture rather than left in its current unused state; carrying dead code that looks functional but has never been trained is a worse outcome than not having built it at all, since it misrepresents what the deployed system actually does. A newcomer taking up this project without that context would reasonably assume the zone selector contributes to the agent's behaviour, when in fact it does not yet.

References

- AFR100. (2025). *Rwanda Eastern Province ROAM and forest landscape restoration strategy 2024–2030*. Winrock International / AFR100. <https://afr100.org>
- Climate Hazards Center. (2025). *Climate Hazards Center Infrared Precipitation with Stations version 3 (CHIRPS v3)* [Data set]. University of California Santa Barbara. <https://doi.org/10.15780/G2JQ0P>
- Dradoum, A., Khelassi, A., & Lachekhab, F. (2025). Intelligent path planning algorithms for UAVs. *Advances in Mechanical Engineering*, 17(1). <https://doi.org/10.1177/16878132251355020>
- The Forefront Magazine. (2025, November 14). *2001–2024: Deforestation rates and statistics in Rwanda*. Retrieved February 2026, from <https://theforefrontmagazine.com/2001-2024-deforestation-rates-and-statistics-in-rwanda/>
- Global Forest Watch. (2024). *Rwanda deforestation rates and statistics* [Dashboard]. World Resources Institute. Retrieved February 2026, from <https://www.globalforestwatch.org/dashboards/country/RWA/>
- Government of Rwanda. (2024). *National Strategy for Transformation 2 (NST2), 2024–2029*. Ministry of Finance and Economic Planning.
- ICRAF. (2022). *Suitable tree species for Rwanda* [Data set]. World Agroforestry Centre (ICRAF). <https://doi.org/10.34725/DVN/RLQV3H>
- IISD. (2024). *Measures to enhance forest conservation and reduce deforestation*. International Institute for Sustainable Development.
- IUCN. (2025, October 27). *Rwanda to plant over 65 million seedlings in 2024–2025*. IUCN.
- Luo, D., Maheshwari, A., Danieleescu, A., Li, J., Yang, Y., Tao, Y., Sun, L., Patel, D. K., Wang, G., Yang, S., Zhang, T., & Yao, L. (2023). Autonomous self-burying seed carriers for aerial seeding. *Nature*, 614(7948), 463–470. <https://doi.org/10.1038/s41586-022-05656-3>
- Margapuri, V. (2025). Prompt-informed reinforcement learning for visual coverage path planning. *Proceedings of the British Machine Vision Conference (BMVC 2025)*.
- Microsoft. (2025). *Playwright documentation*. Microsoft Corporation.
- Mongabay. (2023, July). *New tree tech: Cutting-edge drones give reforestation a helping hand*. Mongabay.
- ONNX. (2025). *Open Neural Network Exchange documentation*. Linux Foundation AI & Data.
- Prisma. (2025). *Prisma ORM documentation*. Prisma Data, Inc.

- Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-Baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268), 1–8. <http://jmlr.org/papers/v22/20-1364.html>
- Reuters. (2024, January 12). *In Brazil, drones take flight in Rio in high-tech reforestation push*. Reuters.
- Robinson, J. M., Harrison, P. A., Mavoa, S., & Breed, M. F. (2022). Existing and emerging uses of drones in restoration ecology. *Methods in Ecology and Evolution*, 13(9), 1899–1911. <https://doi.org/10.1111/2041-210X.13912>
- Siedler, P. D. (2022). *Dynamic collaborative multi-agent reinforcement learning communication for autonomous drone reforestation*. arXiv. <https://arxiv.org/abs/2211.15414>
- Stamatopoulos, I., Le, T. C., & Daver, F. (2024). UAV-assisted seeding and monitoring of reforestation sites: A review. *Australian Forestry*, 87(2), 90–98. <https://doi.org/10.1080/00049158.2024.2343516>
- Theile, M., Zapata Rodriguez, A. R., Caccamo, M., & Sangiovanni-Vincentelli, A. L. (2025). *Continuous world coverage path planning for fixed-wing UAVs using deep reinforcement learning*. arXiv. <https://arxiv.org/abs/2505.08382>
- UNEP. (2023). *Forest information: Rwanda*. United Nations Environment Programme. <https://dicf.unepgrid.ch/rwanda/forest>
- UNEP-WCMC & IUCN. (2026). *Protected Planet: The World Database on Protected Areas (WDPA)* [March 2026 release]. <https://www.protectedplanet.net/country/RWA>
- Unity Technologies. (2025). *Unity Inference Engine (Sentis) documentation*. Unity Technologies.
- Vercel. (2025). *Next.js documentation*. Vercel Inc.